BỘ GIÁO DỤC VÀ ĐÀO TẠO

ĐẠI HỌC KINH TẾ TP.HỒ CHÍ MINH

—----------

UEH
UNIVERSITY

BÁO CÁO

MÔN HỆ HỖ TRỢ QUẢN TRỊ THÔNG MINH

Giáo viên: Hồ Thị Thanh Tuyến

Nhóm: 2

TÊN ĐỀ TÀI: SỬ DỤNG CÔNG NGHỆ KẾT HỢP VỚI NỀN TẢNG BIG DATA TRONG VIỆC PHÂN TÍCH DỮ LIỆU CLICKSTREAM VÀ ĐỀ XUẤT SẢN PHẨM PHÙ HỢP CHO NGƯỜI DÙNG TRÊN WEBSITE THƯƠNG MẠI ĐIỆN TỬ

Thành viên nhóm - MSSV

Hoàng Lưu Hoài Bảo - 31231023697

Đỗ Trần Đan Trường - 31231020272

Trần Trọng Nguyễn - 31231024329

Nguyễn Lê Nhật Đăng Quang - 31231023355

Nguyễn Hà My - 31231027835

PHÂN CÔNG CÔNG VIỆC

STT	Họ và tên	MSSV	Nhiệm vụ cụ thể	Mức độ hoàn thành
1	Nguyễn Lê Nhật Đăng Quang *(Nhóm Trưởng)*	31231023355	- Tìm đề tài - Viết chương trình python và chạy dữ liệu phân tích đề xuất - Viết báo cáo phần kết quả phân tích đề xuất - Quay video demo	100%
2	Nguyễn Hà My	31231027835	- Viết báo cáo phần kết luận, mô tả thuật toán - Chỉnh sửa tổng quát báo cáo	100%
3	Đỗ Trần Đan Trường	31231020272	- Viết báo cáo phần cơ sở lý thuyết Big Data, và công nghệ Big Data, phần kết quả trực quan hóa - Trình bày, chỉnh sửa tổng quát Slide thuyết trình	100%
4	Trần Trọng Nguyễn	31231024329	- Viết chương trình python và chạy dữ liệu phân tích trực quan hóa đưa lên Streamlit - Soạn script thuyết trình báo cáo	100%
5	Hoàng Lưu Hoài Bảo	31231023697	- Xây dựng sườn báo cáo - Chỉnh sửa tổng quát báo cáo - Viết báo cáo phần tổng quan đề tài và mô tả công cụ	100%

TÓM TẮT ĐỀ TÀI

Đề tài nghiên cứu việc ứng dụng công nghệ hiện đại kết hợp với nền tảng Big Data để phân tích dữ liệu clickstream và đề xuất sản phẩm phù hợp cho người dùng trên website thương mại điện tử. Clickstream là tập hợp dữ liệu phản ánh hành vi duyệt web của người dùng, bao gồm các lượt nhấp chuột, thời gian tương tác, hành trình di chuyển giữa các trang và các hành động cụ thể trên website. Thông qua việc khai thác Big Data, hệ thống có thể xử lý và phân tích lượng dữ liệu khổng lồ theo thời gian thực, sử dụng các thuật toán học máy (machine learning), kết hợp với các công cụ để phân tích, lưu trữ dữ liệu như MongoDB, Python, PySpark nhằm nhận diện sở thích, xu hướng mua sắm cũng như dự đoán nhu cầu của khách hàng một cách chính xác, từ đó đưa ra đề xuất phù hợp dựa trên dữ liệu khách hàng. Hệ thống đề xuất sản phẩm cá nhân hóa giúp nâng cao trải nghiệm người dùng, gia tăng tỷ lệ chuyển đổi và tối đa hóa doanh thu cho doanh nghiệp thương mại điện tử. Nghiên cứu này sẽ tập trung vào việc thu thập, xử lý và phân tích dữ liệu clickstream, đồng thời xây dựng mô hình gợi ý sản phẩm dựa trên hành vi người dùng. Bên cạnh đó, đề tài cũng đánh giá hiệu quả của các mô hình đề xuất trong việc tối ưu hóa chiến lược kinh doanh và nâng cao sự hài lòng của khách hàng.

MỤC LỤC

PHÂN CÔNG CÔNG VIỆC....3
TÓM TẮT ĐỀ TÀI....3
DANH MỤC HÌNH ẢNH....3
DANH MỤC BẢNG BIỂU....3
DANH MỤC TỪ VIẾT TẮT....3
CHƯƠNG 1: GIỚI THIỆU ĐỀ TÀI, BÀI TOÁN....4
1.1. Lý do chọn đề tài....4
1.2. Mục đích, mục tiêu của đề tài....4
1.3. Đối tượng và phạm vi của đề tài....5
1.4. Phương pháp thực hiện....5
1.5. Đóng góp của đề tài....5
CHƯƠNG 2: MÔ TẢ CÔNG CỤ, BỘ DỮ LIỆU....6
2.1. Định nghĩa Big Data....6
2.2. Công nghệ Big Data....6
2.2.1. MapReduce....6
2.2.1.1. Giới thiệu về MapReduce....6
2.2.1.2. Cách thức hoạt động của MapReduce....7
2.2.2. Hadoop....8
2.2.2.1. Giới thiệu về Hadoop....8
2.2.2.2. Cách thức hoạt động của Hadoop....8
2.2.2.3. Cách thức hoạt động của Hadoop....9
2.2.3. NoSQL....10
2.2.3.1. Giới thiệu về NoSQL....10
2.2.3.2. Các loại cơ sở dữ liệu NoSQL....10
2.3. Công cụ xử lý dữ liệu MongoDB....11
2.3.1. Các thuật ngữ trong MongoDB....12
2.2.2. Mô hình triển khai MongoDB....12
2.2.2.1. Mô hình....12
2.2.2.2. Hạ tầng triển khai....14
2.2.2.2.1. Sử dụng Cloud (MongoDB Atlas):....14
2.2.2.2.2. Triển khai On-premise....15
2.2.3. Kiến trúc hạ tầng....16
2.2.4. Làm việc với dữ liệu trong MongoDB....16
2.2.4.1. Kết nối đến MongoDB....16
2.2.5. Cách tối ưu MongoDB....17
2.3. Ngôn ngữ lập trình Python....18
2.3.1. Lịch sử hình thành....18
2.3.2. Ứng dụng của Python....18
2.3.3. Ưu điểm và nhược điểm của Python....19
2.4. Công cụ xử lý Big Data PySpark....20

2.4.1. PySpark là gì?....20
2.4.2. Ứng dụng thực tế của PySpark....20
2.4.3. Ưu điểm và nhược điểm của PySpark....21
2.5. Mô tả bộ dữ liệu....22
2.5.1. Giới thiệu bộ dữ liệu....22
2.5.2. Tại sao cần sử dụng Big Data để giải quyết bài toán....24
CHƯƠNG 3: MÔ TẢ THUẬT TOÁN, DEMO ỨNG DỤNG....25
3.1. Mô tả thuật toán....25
3.1.1. Thu thập và lưu trữ dữ liệu....25
3.1.2. Tiền xử lý dữ liệu....26
3.1.3. Trực quan hóa dữ liệu và đưa lên Streamlit....28
3.1.4. Phân tích dữ liệu và đưa ra đề xuất bằng PySpark....32
3.2. Demo ứng dụng....38
3.3. Phân tích, giải thích kết quả thu được....39
3.3.1. Phân tích, giải thích kết quả phần trực quan hóa....39
3.3.2. Kết quả phân tích đề xuất bằng PySpark....48
CHƯƠNG 4: KẾT LUẬN BÀI TOÁN....49
4.1. Kết luận bài toán....49
4.2. Đánh giá ưu điểm, nhược điểm....49
4.2.1. Ưu điểm....49
4.2.2. Nhược điểm....50
4.3. Hướng phát triển của đề tài....50

DANH MỤC HÌNH ẢNH

Hình 1: Sơ đồ các thức hoạt động của MapReduce..... 12
Hình 2: Các loại cơ sở dữ liệu phổ biến và đặc biệt của NoSQL.....15
Hình 3: So sánh một vài thuật ngữ trong MongoDB và RDBMS.....17
Hình 4: Minh họa mô hình Standalone..... 17
Hình 5: Minh họa cho mô hình Replication.....18
Hình 6: Minh họa cho mô hình Sharding.....19
Hình 7: Trang chủ MongoDB Atlas..... 19
Hình 8: Triển khai on-premise trên MongoDB.....20
Hình 9: Lịch sử hình thành Python.....23
Hình 10: Ứng dụng thực tế của Python.....23
Hình 11: Công cụ PySpark.....25
Hình 12: Ứng dụng của PySpark.....26
Hình 13: Dữ liệu được lưu trữ dưới dạng CSV.....31
Hình 14: Dữ liệu lưu trữ dưới dạng JSON trong MongoDB.....31
Hình 15: Bảng kết quả trực quan hóa tổng quan.....44
Hình 16: Biểu đồ minh họa sự phân bố lượt xem dựa trên kiểu chụp.....45
Hình 17: Biểu đồ minh họa số lượt truy cập thực sự theo quốc gia.....45
Hình 18: Biểu đồ minh họa số lần click trung bình mỗi phiên theo quốc gia.....46
Hình 19: Biểu đồ minh họa số lượng phiên truy cập theo tháng.....47
Hình 20: Biểu đồ minh họa số lượng truy cập theo các phân loại giá cả khác nhau.....47
Hình 21: Biểu đồ minh họa tổng lượt xem theo từng danh mục sản phẩm.....48
Hình 22: Biểu đồ minh họa số lượt xem của các danh mục sản phẩm trên từng trang.....49
Hình 23: Biểu đồ minh họa gian hàng thu hút và màu sắc sản phẩm phổ biến nhất.....50
Hình 24: Biểu đồ minh họa số lượng đơn đặt hàng theo màu sắc sản phẩm.....51
Hình 25: Biểu đồ minh họa mối quan hệ giữa các nước và số ngày theo dõi..... 52
Hình 26: Kết quả thuật toán MapReduce..... 54
Hình 27: Kết quả thuật toán đề xuất dựa trên MapReduce..... 54

DANH MỤC BẢNG BIỂU

Bảng 2.1. Giới thiệu các biến có trong tập dữ liệu..... 27

CHƯƠNG 1: GIỚI THIỆU ĐỀ TÀI, BÀI TOÁN

1.1. Lý do chọn đề tài

Trong thời kỳ cạnh tranh kinh doanh và sự bùng nổ của kỹ thuật số, việc tận dụng hết tất cả các nguồn tài nguyên dữ liệu về người dùng để tối ưu hóa trải nghiệm người dùng và tăng tỷ lệ chuyển đổi mua hàng là một trong những ưu tiên hàng đầu của các doanh nghiệp. Theo báo cáo của Forbes Advisor (2023), doanh thu thương mại điện tử toàn cầu dự kiến sẽ đạt 6,3 nghìn tỷ USD vào năm 2024, với mức tăng trưởng trung bình 8,9% mỗi năm. Tuy nhiên, một trong những thách thức lớn nhất đối với các nền tảng thương mại điện tử là giúp người dùng tìm thấy sản phẩm phù hợp một cách nhanh chóng. Theo một nghiên cứu của Baymard Institute (2022), 69% giỏ hàng trực tuyến bị bỏ dở vì người dùng không tìm thấy sản phẩm phù hợp hoặc mất quá nhiều thời gian để ra quyết định mua hàng.

Trong bối cảnh đó, việc sử dụng trí tuệ nhân tạo và máy học để phân tích dữ liệu người dùng đang ngày càng trở thành một xu hướng thiết yếu. Các doanh nghiệp không chỉ dừng lại ở việc thu thập dữ liệu, mà còn cần phải biết cách diễn giải và áp dụng dữ liệu đó vào các chiến lược kinh doanh cụ thể. Chẳng hạn, thông qua việc phân tích dữ liệu clickstream - dữ liệu ghi lại hành vi duyệt web và tương tác của người dùng trên trang web, doanh nghiệp có thể xác định được những điểm dừng chân quan trọng của người dùng trên trang web, từ đó tối ưu hóa giao diện người dùng và cải thiện hành trình mua sắm. Ngoài ra, việc tích hợp dữ liệu từ các nguồn khác nhau, chẳng hạn như mạng xã hội, lịch sử mua hàng, và phản hồi của khách hàng, cũng giúp nâng cao độ chính xác của các hệ thống đề xuất. Theo một nghiên cứu của Gartner (2022), các công ty áp dụng phân tích dữ liệu toàn diện có khả năng gia tăng lợi nhuận cao hơn 20% so với các công ty không sử dụng. Điều này cho thấy rõ ràng rằng việc đầu tư vào các công nghệ xử lý và phân tích dữ liệu không chỉ là một sự lựa chọn, mà còn là một yếu tố quyết định sự thành công của các doanh nghiệp trong thời đại số hóa hiện nay. Tương lai của thương mại điện tử không chỉ nằm ở việc có được một sản phẩm tốt, mà còn ở khả năng đưa sản phẩm đó đến đúng khách hàng vào đúng thời điểm, nhờ vào sự hỗ trợ của công nghệ tiên tiến.

1.2. Mục đích, mục tiêu của đề tài

Trong bối cảnh phát triển mạnh mẽ của thương mại điện tử, việc hiểu rõ hành vi và sở thích của người dùng là yếu tố then chốt giúp doanh nghiệp nâng cao trải nghiệm người dùng và tối ưu hóa chiến lược kinh doanh của mình. Mục tiêu của đề tài là nghiên cứu và phát triển một hệ thống phân tích dữ liệu dựa trên nền tảng Big Data để thu thập và xử lý dữ liệu clickstream từ các website thương mại điện tử. Bằng việc áp dụng các công nghệ tiên tiến như trí tuệ nhân tạo và học máy, hệ thống này sẽ giúp phân tích sâu sắc hơn về hành vi người dùng, từ đó đưa ra các đề xuất sản phẩm phù hợp và cá nhân hóa hơn cho từng khách hàng. Dữ liệu clickstream, vốn là tập hợp các hành động của người dùng trên website như việc nhấp chuột, thời gian truy cập, và đường dẫn đã xem, sẽ được xử lý để xác định các mô hình hành vi và xu hướng tiêu dùng. Phân tích dữ liệu này không chỉ giúp doanh nghiệp hiểu rõ hơn về nhu cầu của khách hàng mà còn tối ưu hóa quy trình tiếp thị và quản lý kho hàng, từ đó cải thiện

hiệu suất kinh doanh. Đề tài này kỳ vọng sẽ tạo ra một bước đột phá trong việc cá nhân hóa trải nghiệm mua sắm trực tuyến, đồng thời mở ra những hướng đi mới cho việc ứng dụng công nghệ trong kinh doanh hiện đại.

1.3. Đối tượng và phạm vi của đề tài

Đề tài "Sử dụng công nghệ kết hợp với nền tảng Big Data trong việc phân tích dữ liệu clickstream và đề xuất sản phẩm phù hợp cho người dùng trên website thương mại điện tử" tập trung vào việc áp dụng công nghệ tiên tiến để cải thiện trải nghiệm mua sắm trực tuyến.

+ **Đối tượng nghiên cứu**: các doanh nghiệp thương mại điện tử, các nhà phát triển công nghệ thông tin, và các chuyên gia phân tích dữ liệu.
+ **Phạm vi của đề tài**: Nhóm sẽ sử dụng các công cụ và kỹ thuật Big Data để thu thập và phân tích dữ liệu clickstream – một dạng dữ liệu ghi lại hành vi của người dùng khi họ tương tác với website.

Từ đó, hệ thống sẽ đưa ra các gợi ý sản phẩm cá nhân hóa dựa trên sở thích và hành vi tiêu dùng của từng người dùng, nhằm tối ưu hóa trải nghiệm mua sắm và gia tăng doanh thu cho doanh nghiệp. Đề tài này không chỉ giới hạn ở khía cạnh kỹ thuật mà còn mở rộng đến việc đánh giá hiệu quả của các mô hình đề xuất sản phẩm và tác động của chúng đối với sự hài lòng của khách hàng và lợi ích kinh tế của doanh nghiệp.

1.4. Phương pháp thực hiện

+ **Phương pháp thu thập dữ liệu:** Sử dụng dữ liệu thực tế từ sàn thương mại điện tử thông qua công cụ Kaggle
+ **Phương pháp phân tích:** Sử dụng các công cụ Business Intelligence liên quan đến Big Data như MongoDB, Python,... để trực quan hóa và phân tích dữ liệu, từ đó đưa ra các giải pháp cải thiện hoạt động chuỗi cung ứng.

1.5. Đóng góp của đề tài

Đề tài **"Sử dụng công nghệ kết hợp với nền tảng Big Data trong việc phân tích dữ liệu clickstream và đề xuất sản phẩm phù hợp cho người dùng trên website thương mại điện tử"** mang lại nhiều đóng góp quan trọng cho lĩnh vực thương mại điện tử. Đầu tiên, nó cung cấp một phương pháp tiếp cận tiên tiến để phân tích hành vi người dùng thông qua dữ liệu clickstream, giúp các doanh nghiệp hiểu rõ hơn về thói quen mua sắm và sở thích của khách hàng. Bên cạnh đó việc áp dụng công nghệ Big Data giúp xử lý và phân tích một lượng lớn dữ liệu trong thời gian thực, từ đó đưa ra các quyết định kinh doanh nhanh chóng và chính xác hơn. Ngoài ra, hệ thống đề xuất sản phẩm được tối ưu hóa dựa trên dữ liệu thu thập được, không chỉ nâng cao trải nghiệm người dùng mà còn tăng khả năng bán hàng và doanh thu cho các doanh nghiệp. Cuối cùng là đề tài này còn mở ra cơ hội cho việc phát triển các ứng dụng và giải pháp công nghệ mới, nhằm tối ưu hóa hơn nữa hiệu quả của các nền tảng thương mại điện tử trong bối cảnh cạnh tranh ngày càng gay gắt. Việc tích hợp công nghệ hiện đại với Big Data không chỉ là xu hướng tất yếu mà còn là yếu tố then

chốt giúp các doanh nghiệp nâng cao vị thế và khả năng cạnh tranh trên thị trường toàn cầu. Sự kết hợp này không những giúp tối ưu hóa quy trình kinh doanh, mà còn mở rộng phạm vi tiếp cận khách hàng, đưa ra các chiến lược marketing thông minh và hiệu quả hơn. Đề tài không chỉ góp phần thúc đẩy sự phát triển của ngành thương mại điện tử mà còn tạo nền tảng cho các nghiên cứu và ứng dụng công nghệ trong tương lai.

CHƯƠNG 2: MÔ TẢ CÔNG CỤ, BỘ DỮ LIỆU

2.1. Định nghĩa Big Data

Big Data là một thuật ngữ dùng để chỉ khối lượng dữ liệu lớn và phức tạp đến mức khó khăn trong việc xử lý bằng các công cụ quản lý dữ liệu truyền thống. Theo Gartner, Big Data được định nghĩa là *"khối lượng dữ liệu cao, tốc độ xử lý nhanh và đa dạng, yêu cầu các hình thức xử lý thông tin sáng tạo và hiệu quả cao để tăng cường hiểu biết, ra quyết định và tự động hóa quy trình."* Big Data không chỉ bao gồm dữ liệu có cấu trúc mà còn cả dữ liệu phi cấu trúc, đòi hỏi sự kết hợp của các công nghệ tiên tiến và phân tích dữ liệu để trích xuất thông tin hữu ích. Các đặc điểm nổi bật của Big Data thường được mô tả bằng ba chữ V: **Volume** *(Khối lượng)*, **Velocity** *(Tốc độ)*, và **Variety** *(Đa dạng)*. Sự phát triển nhanh chóng của Internet, mạng xã hội, và các thiết bị kết nối đã làm gia tăng lượng dữ liệu khổng lồ được tạo ra mỗi ngày, thúc đẩy nhu cầu xử lý Big Data để cải thiện quyết định kinh doanh và tối ưu hóa hoạt động trong nhiều lĩnh vực khác nhau, từ y tế, tài chính đến công nghiệp và chính phủ

2.2. Công nghệ Big Data

Mặc dù có nhiều phương pháp khác nhau để xử lý và phân tích dữ liệu lớn, phần lớn chúng đều có những đặc điểm chung (Delen et al., 2018). Cụ thể, chúng sử dụng khả năng lưu trữ dữ liệu phi quan hệ để xử lý dữ liệu phi cấu trúc và bán cấu trúc, tận dụng phần cứng phổ thông để mở rộng quy mô và thực hiện xử lý song song, đồng thời áp dụng các công nghệ phân tích nâng cao và trực quan hóa dữ liệu để cung cấp thông tin chi tiết cho người dùng. MapReduce, Hadoop và NoSQL là ba công nghệ Big Data nổi bật nhất, được nhiều người tin rằng sẽ cách mạng hóa lĩnh vực phân tích kinh doanh và quản lý dữ liệu.

2.2.1. MapReduce

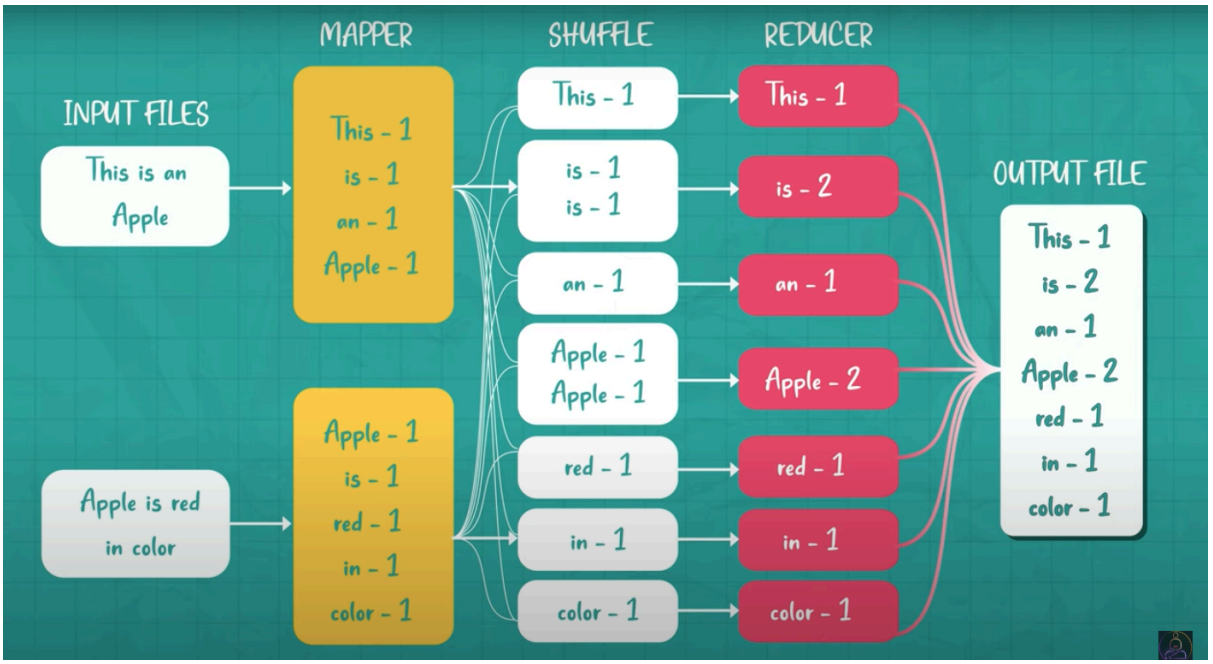
2.2.1.1. Giới thiệu về MapReduce

MapReduce là mô hình lập trình do Google phát triển để xử lý dữ liệu lớn theo cách phân tán trên nhiều máy tính, giúp tối ưu hóa hiệu suất tính toán bằng cách xử lý song song. Mô hình này được giới thiệu vào năm 2004 trong bài báo "MapReduce: Simplified Data Processing on Large Clusters" của Jeffrey Dean và Sanjay Ghemawat, nhằm giải quyết bài toán xử lý dữ liệu lớn trên cụm máy chủ phân tán, từ đó nâng cao tốc độ xử lý so với các phương pháp truyền thống. Nhờ khả năng mở rộng và xử lý song song, MapReduce trở thành nền tảng quan trọng trong các hệ thống xử lý dữ liệu lớn, tiêu biểu như Apache Hadoop – một nền tảng mã nguồn mở áp dụng mô hình này để lưu trữ và xử lý dữ liệu phi cấu trúc hiệu quả. Và với ưu điểm về tốc độ, khả năng mở rộng và giảm tải tài nguyên, mô hình MapReduce đã góp phần quan trọng vào sự phát triển của công nghệ dữ liệu lớn, đặc biệt trong các lĩnh vực như tìm kiếm, phân tích dữ liệu và trí tuệ nhân tạo.

2.2.1.2. Cách thức hoạt động của MapReduce

MapReduce hoạt động dựa trên nguyên tắc chia nhỏ dữ liệu và xử lý song song. Quá trình này diễn ra trong ba giai đoạn chính: Map, Shuffle and Sort, và Reduce. Dữ liệu đầu vào được chia thành nhiều phần nhỏ và được xử lý đồng thời trên nhiều máy tính trong một hệ thống phân tán.

Trong giai đoạn Map, dữ liệu đầu vào được chia thành các phần nhỏ gọi là input splits để xử lý song song. Mỗi phần dữ liệu này sau đó được xử lý bởi một hàm Map, có nhiệm vụ chuyển đổi dữ liệu thành các cặp (key, value) nhằm chuẩn bị cho bước xử lý tiếp theo. Một ví dụ rõ nhất là trong bài toán đếm số lần xuất hiện của từ trong một văn bản, hàm Map sẽ quét qua từng từ và tạo ra các cặp (từ, 1), biểu thị rằng mỗi từ xuất hiện một lần. Những cặp này sau đó sẽ được chuyển đến giai đoạn Reduce để tổng hợp kết quả.



Hình 1: Sơ đồ các thức hoạt động của MapReduce

Sau khi dữ liệu được xử lý bởi hàm Map, các cặp (key, value) sẽ được sắp xếp theo key để chuẩn bị cho bước tổng hợp. Trong quá trình này, các cặp có cùng key sẽ được nhóm lại và gửi đến cùng một bộ xử lý Reduce. Việc trộn và sắp xếp giúp đảm bảo rằng tất cả các giá trị liên quan đến một key nhất định đều được xử lý một cách hiệu quả, giúp tối ưu hóa quá trình tính toán và tổng hợp dữ liệu trong giai đoạn tiếp theo.

Trong giai đoạn Reduce, dữ liệu sau khi đã được sắp xếp và gom nhóm sẽ được truyền đến các hàm Reduce để xử lý. Tại đây, hàm Reduce thực hiện tổng hợp hoặc

tính toán trên các giá trị tương ứng với từng key, giúp tạo ra kết quả cuối cùng. Sau khi xử lý xong, dữ liệu được ghi vào hệ thống lưu trữ, chẳng hạn như HDFS (Hadoop Distributed File System). Một ví dụ rõ nhất là trong bài toán đếm số lần xuất hiện của từ, giai đoạn Reduce sẽ cộng tất cả các giá trị 1 của từng từ, tạo ra đầu ra dưới dạng (từ, tổng số lần xuất hiện), cung cấp thông tin thống kê hữu ích về tần suất của các từ trong văn bản.

2.2.2. Hadoop

2.2.2.1. Giới thiệu về Hadoop

Hadoop là một nền tảng mã nguồn mở được thiết kế để lưu trữ và xử lý dữ liệu lớn theo mô hình phân tán. Bằng cách sử dụng các cụm máy tính và xử lý song song, Hadoop giúp tăng tốc độ xử lý dữ liệu và tối ưu hóa tài nguyên hệ thống. Nền tảng này được xây dựng dựa trên mô hình MapReduce và hệ thống tệp phân tán HDFS (Hadoop Distributed File System), giúp quản lý và phân tích dữ liệu lớn một cách hiệu quả. Hadoop ra đời vào năm 2006 do Doug Cutting và Mike Cafarella phát triển khi họ làm việc trên dự án Nutch – một công cụ tìm kiếm mã nguồn mở. Dự án này được truyền cảm hứng từ bài báo "MapReduce: Simplified Data Processing on Large Clusters" của Google, trong đó mô tả cách xử lý dữ liệu lớn theo mô hình phân tán. Nhờ khả năng mở rộng linh hoạt, Hadoop nhanh chóng trở thành một phần quan trọng của Apache Software Foundation và phát triển thành nền tảng xử lý dữ liệu lớn phổ biến trên toàn thế giới. Với khả năng xử lý dữ liệu phi cấu trúc, hỗ trợ nhiều công cụ phân tích và tính linh hoạt cao, Hadoop đã trở thành giải pháp quan trọng trong nhiều lĩnh vực như công nghệ tài chính, thương mại điện tử, trí tuệ nhân tạo và khoa học dữ liệu.

2.2.2.2. Cách thức hoạt động của Hadoop

Hadoop hoạt động theo nguyên tắc phân tán dữ liệu và xử lý song song trên nhiều máy chủ. Hadoop được xây dựng dựa trên ba phần chính là Hadoop Distributes FileSystem (HDFS), YARN và MapReduce. Trong đó:

- **HDFS (Hadoop Distributed File System)**: HDFS là hệ thống file phân tán được sử dụng để lưu trữ dữ liệu. Nó phân chia dữ liệu thành các khối và phân tán chúng trên nhiều máy tính. HDFS có độ tin cậy cao và thích hợp cho việc lưu trữ khối lượng dữ liệu lớn.
- **YARN (Yet Another Resource Negotiator)**: YARN là một khung phần mềm quản lý và phân bổ tài nguyên để vận hành các ứng dụng trên Hadoop. Nó cung cấp các dịch vụ như quản lý tài nguyên, lên lịch và giám sát cho các ứng dụng chạy trên Hadoop.

- **MapReduce**: MapReduce là khung mô hình lập trình để xử lý và tính toán trên số lượng dữ liệu lớn trong môi trường phân tán. Nó hoạt động dựa trên 2 pha chính là Map và Reduce. Pha Map sẽ chia nhỏ dữ liệu thành các cặp key-value, sau đó pha Reduce sẽ nhóm các cặp key-value lại với nhau để tính toán kết quả cuối cùng.

2.2.2.3. Cách thức hoạt động của Hadoop

Quá trình hoạt động của Hadoop có thể chia thành ba giai đoạn chính: Lưu trữ dữ liệu, Xử lý dữ liệu, và Xuất kết quả.

Trong giai đoạn đầu tiên, giai đoạn lưu trữ, dữ liệu đầu vào được chia thành các khối nhỏ (blocks) và phân phối trên nhiều máy chủ trong hệ thống HDFS (Hadoop Distributed File System). Hệ thống này hoạt động dựa trên hai thành phần chính: NameNode và DataNodes. NameNode chịu trách nhiệm quản lý metadata (siêu dữ liệu), bao gồm vị trí và trạng thái của các khối dữ liệu, trong khi DataNodes đảm nhận việc lưu trữ dữ liệu thực tế trên các máy chủ khác nhau. Để đảm bảo tính sẵn sàng và khả năng chịu lỗi, HDFS sử dụng cơ chế sao chép dữ liệu (replication), giúp bảo vệ dữ liệu trước các sự cố mất mát và tăng cường độ tin cậy của hệ thống.

Khi có một tác vụ xử lý dữ liệu, Hadoop sử dụng mô hình MapReduce để thực thi trên nhiều nút tính toán trong hệ thống phân tán. Quá trình này bao gồm ba giai đoạn chính. Giai đoạn Map (Ánh xạ): Dữ liệu được chia thành các phần nhỏ, mỗi phần được xử lý độc lập và tạo ra các cặp (key, value). Giai đoạn Shuffle and Sort (Trộn và Sắp xếp): Các cặp dữ liệu được nhóm lại theo key, chuẩn bị cho quá trình tổng hợp. Cuối cùng, Giai đoạn Reduce (Tổng hợp): Các dữ liệu đã được nhóm lại sẽ được xử lý và tạo ra kết quả cuối cùng, sau đó lưu trữ vào hệ thống.

Sau khi hoàn thành giai đoạn Reduce, kết quả được lưu trữ trong HDFS hoặc các hệ thống lưu trữ khác để phục vụ cho quá trình phân tích tiếp theo. Dữ liệu đầu ra có thể được sử dụng để tạo báo cáo, hỗ trợ ra quyết định hoặc tích hợp vào các ứng dụng như Machine Learning và Business Intelligence, giúp khai thác giá trị từ dữ liệu một cách hiệu quả.

2.2.3. NoSQL

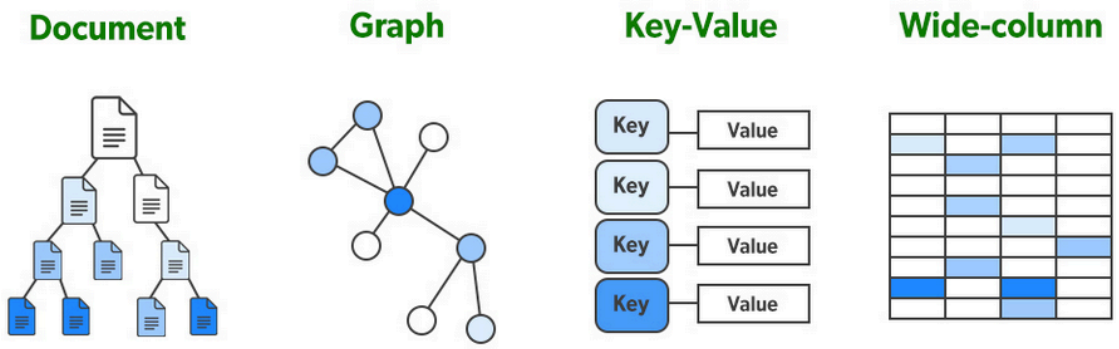
2.2.3.1. Giới thiệu về NoSQL

NoSQL (Not Only SQL) là hệ quản trị cơ sở dữ liệu không tuân theo mô hình quan hệ truyền thống, được thiết kế để lưu trữ và xử lý dữ liệu lớn một cách linh hoạt với khả năng mở rộng quy mô ngang và hiệu suất cao. Khác với cơ sở dữ liệu quan hệ (RDBMS), NoSQL không yêu cầu lược đồ cố định, cho phép lưu trữ dữ liệu dưới

nhiều mô hình khác nhau như key-value, document, column-family và graph. Công nghệ này ra đời vào cuối những năm 2000 nhằm khắc phục các hạn chế của RDBMS trong việc xử lý dữ liệu lớn và mở rộng quy mô. Khi các ứng dụng web, mạng xã hội và hệ thống dữ liệu phi cấu trúc phát triển mạnh, các công ty công nghệ lớn như Google, Amazon, Facebook cần một giải pháp mới để quản lý dữ liệu hiệu quả hơn. Từ đó, hàng loạt hệ thống NoSQL như MongoDB, Cassandra, Redis và Neo4j được phát triển, giúp doanh nghiệp lưu trữ và xử lý dữ liệu theo cách tối ưu hơn. Nhờ khả năng linh hoạt, tốc độ xử lý cao và khả năng thích ứng với dữ liệu phi cấu trúc, NoSQL đã trở thành lựa chọn phổ biến trong các lĩnh vực như thương mại điện tử, phân tích dữ liệu lớn, trí tuệ nhân tạo và các hệ thống thời gian thực.

2.2.3.2. Các loại cơ sở dữ liệu NoSQL

Cơ sở dữ liệu NoSQL bao gồm nhiều loại khác nhau, mỗi loại có đặc điểm và ứng dụng riêng nhằm đáp ứng nhu cầu lưu trữ và xử lý dữ liệu trong nhiều lĩnh vực khác nhau. Các loại cơ sở dữ liệu phổ biến và đặc biệt của NoSQL bao gồm: cơ sở dữ liệu hướng tài liệu, cơ sở dữ liệu hướng cột, cơ sở dữ liệu khóa-giá trị và cơ sở dữ liệu đồ thị.



Hình 2: Các loại cơ sở dữ liệu phổ biến và đặc biệt của NoSQL

Cơ sở dữ liệu hướng tài liệu lưu trữ dữ liệu dưới dạng các tài liệu có cấu trúc linh hoạt, thường sử dụng các định dạng như JSON hoặc XML. Mỗi tài liệu có thể chứa nhiều trường dữ liệu khác nhau mà không cần tuân theo một cấu trúc cố định, giúp hệ thống dễ dàng mở rộng và thay đổi theo nhu cầu. Loại cơ sở dữ liệu này phù hợp với các ứng dụng có cấu trúc dữ liệu linh hoạt và thay đổi thường xuyên, chẳng hạn như hệ thống quản lý nội dung (CMS), blog, hoặc hệ thống quản lý người dùng.

Các hệ quản trị cơ sở dữ liệu phổ biến trong nhóm này bao gồm MongoDB và CouchDB.

Cơ sở dữ liệu hướng cột lưu trữ dữ liệu theo cột thay vì hàng, giúp tối ưu hóa việc truy xuất dữ liệu theo từng trường cụ thể. Mỗi cột có thể chứa các giá trị thuộc nhiều hàng khác nhau, cho phép hệ thống truy vấn nhanh chóng trên một tập hợp lớn các bản ghi. Loại cơ sở dữ liệu này phù hợp cho các ứng dụng có yêu cầu đọc và ghi dữ liệu theo cột lớn, như phân tích dữ liệu, quản lý thông tin kinh doanh, hoặc lưu trữ lịch sử dữ liệu. Một số hệ quản trị cơ sở dữ liệu phổ biến theo mô hình này là Apache Cassandra và HBase.

Cơ sở dữ liệu khóa-giá trị lưu trữ dữ liệu dưới dạng các cặp (key, value), trong đó mỗi khóa duy nhất tương ứng với một giá trị cụ thể. Hệ thống này có tốc độ truy xuất rất cao vì chỉ cần tìm kiếm theo khóa để lấy dữ liệu mà không cần thực hiện các phép truy vấn phức tạp. Nó thường được sử dụng trong các ứng dụng yêu cầu lưu trữ dữ liệu tạm thời, bộ nhớ đệm (caching), hoặc quản lý phiên người dùng. Những hệ quản trị phổ biến của loại cơ sở dữ liệu này bao gồm Redis và DynamoDB.

Cơ sở dữ liệu đồ thị lưu trữ dữ liệu dưới dạng các đỉnh (nodes) và cạnh (edges), giúp thể hiện mối quan hệ giữa các phần tử một cách trực quan và hiệu quả. Điều này giúp loại cơ sở dữ liệu này trở nên lý tưởng cho các hệ thống có cấu trúc dữ liệu quan hệ phức tạp, chẳng hạn như mạng xã hội, hệ thống phân loại sản phẩm, hoặc quản lý quan hệ khách hàng (CRM). Với khả năng truy vấn nhanh và linh hoạt, cơ sở dữ liệu đồ thị được sử dụng rộng rãi trong các ứng dụng phân tích dữ liệu quan hệ và hệ thống gợi ý. Những hệ quản trị nổi bật trong nhóm này bao gồm Neo4j và ArangoDB.

2.3. Công cụ xử lý dữ liệu MongoDB

MongoDB là một cơ sở dữ liệu mã nguồn mở và là cơ sở dữ liệu NoSQL(*) hàng đầu, được hàng triệu người sử dụng. MongoDB được viết bằng C++. Ngoài ra, MongoDB là một cơ sở dữ liệu đa nền tảng, hoạt động trên các khái niệm Collection và Document, nó cung cấp hiệu suất cao, tính khả dụng cao và khả năng mở rộng dễ dàng.

2.3.1. Các thuật ngữ trong MongoDB

+ **_id –** Là trường bắt buộc có trong mỗi document. Trường _id đại diện cho một giá trị duy nhất trong document MongoDB. Trường _id cũng có thể được hiểu là khóa chính trong document. Nếu thêm mới một document thì MongoDB sẽ tự động sinh ra một _id đại diện cho document đó và là duy nhất trong cơ sở dữ liệu MongoDB.
+ **Collection** – Là nhóm của nhiều document trong MongoDB. Collection có thể được hiểu là một bảng tương ứng trong cơ sở dữ liệu RDBMS (Relational Database Management System). Collection nằm trong một cơ sở dữ liệu duy

nhất. Các collection không phải định nghĩa các cột, các hàng hay kiểu dữ liệu trước.

+ **Cursor** – Đây là một con trỏ đến tập kết quả của một truy vấn. Máy khách có thể lặp qua một con trỏ để lấy kết quả.
+ **Database** – Nơi chứa các Collection, giống với cơ sở dữ liệu RDBMS chúng chứa các bảng. Mỗi Database có một tập tin riêng lưu trữ trên bộ nhớ vật lý. Một máy chủ MongoDB có thể chứa nhiều Database.
+ **Document** – Một bản ghi thuộc một Collection thì được gọi là một Document. Các Document lần lượt bao gồm các trường tên và giá trị.
+ **Field** – Là một cặp name – value trong một document. Một document có thể có không hoặc nhiều trường. Các trường giống các cột ở cơ sở dữ liệu quan hệ.
+ **JSON** – Viết tắt của JavaScript Object Notation. Con người có thể đọc được ở định dạng văn bản đơn giản thể hiện cho các dữ liệu có cấu trúc. Hiện tại JSON đang hỗ trợ rất nhiều ngôn ngữ lập trình.
+ **Index** – Là những cấu trúc dữ liệu đặc biệt, dùng để chứa một phần nhỏ của các tập dữ liệu một cách dễ dàng để quét. Chỉ số lưu trữ giá trị của một fields cụ thể hoặc thiết lập các fields, sắp xếp theo giá trị của các fields này. Index hỗ trợ độ phân tích một cách hiệu quả các truy vấn. Nếu không có chỉ mục, MongoDB sẽ phải quét tất cả các documents của collection để chọn ra những document phù hợp với câu truy vấn. Quá trình quét này là không hiệu quả và yêu cầu MongoDB để xử lý một khối lượng lớn dữ liệu.

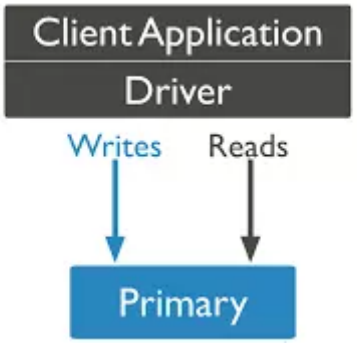
RDBMS	Database	Table	Column	Row	Index
MongoDB	Database	Collection	Field	Document	Index

Hình 3: So sánh một vài thuật ngữ trong MongoDB và RDBMS

2.2.2. Mô hình triển khai MongoDB

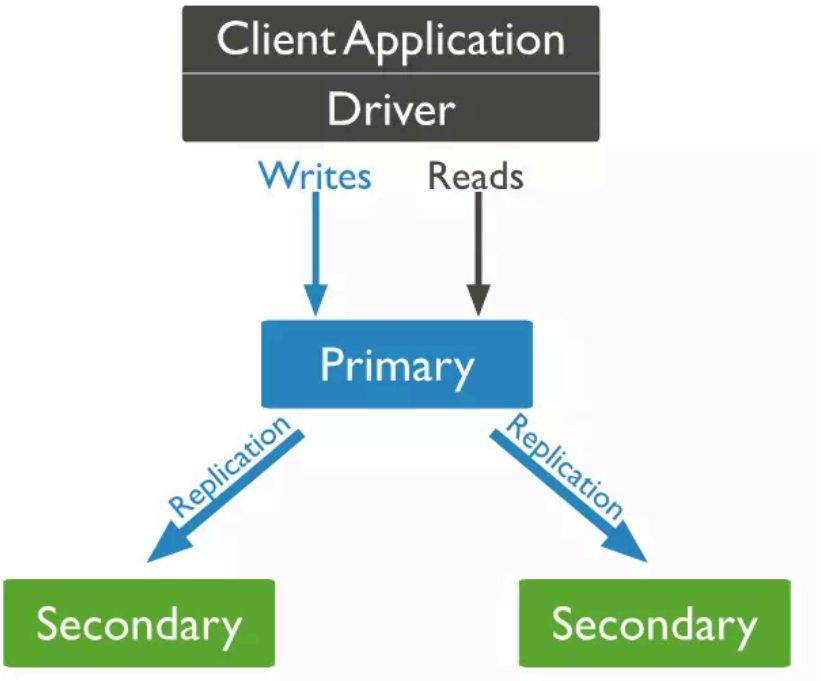
2.2.2.1. Mô hình

Standalone: Đây là triển khai cơ bản nhất, với một instance duy nhất của MongoDB. Mô hình này phù hợp cho môi trường phát triển, thử nghiệm hoặc các ứng dụng nhỏ không yêu cầu tính sẵn sàng cao. Mô hình này bao gồm dễ cài đặt và quản lý tuy nhiên không có khả năng chịu lỗi; nếu server gặp sự cố, dịch vụ sẽ bị gián đoạn. Cũng như không hỗ trợ mở rộng ngang và khả năng bị ảnh hưởng tới hiệu năng do đọc, ghi liên tục và database. Do đó, cần xử lý để tối ưu khi gặp.



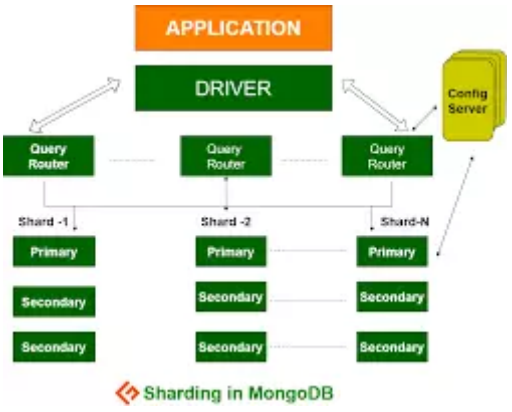
Hình 4: Minh họa mô hình Standalone

Replication: Replication tạo ra một replica set, bao gồm một primary và nhiều secondary nodes. Primary node xử lý các hoạt động ghi, trong khi các secondary nodes duy trì bản sao dữ liệu và có thể xử lý các yêu cầu đọc (tùy cấu hình). Về đặc điểm, mô hình này đã tăng tính sẵn sàng và khả năng chịu lỗi; nếu primary node gặp sự cố, một secondary node sẽ được bầu làm primary mới, cải thiện khả năng đọc bằng cách phân phối tải đọc đến các secondary nodes đồng thời đảm bảo dữ liệu được sao chép đồng bộ giữa các nodes.



Hình 5: Minh họa cho mô hình Replication

Sharding: Sharding phân chia dữ liệu thành các phần nhỏ gọi là shards, mỗi shard là một replica set. Mô hình này cho phép MongoDB mở rộng theo chiều ngang, xử lý các tập dữ liệu lớn và tải công việc cao. Đặc điểm của mô hình này gồm phân phối dữ liệu và tải công việc trên nhiều máy chủ, cải thiện hiệu suất và khả năng mở rộng tuy nhiên yêu cầu cấu hình phức tạp hơn, bao gồm các config servers để lưu trữ metadata và mongos để định tuyến các yêu cầu nên thích hợp cho các ứng dụng với yêu cầu lưu trữ và xử lý dữ liệu lớn.



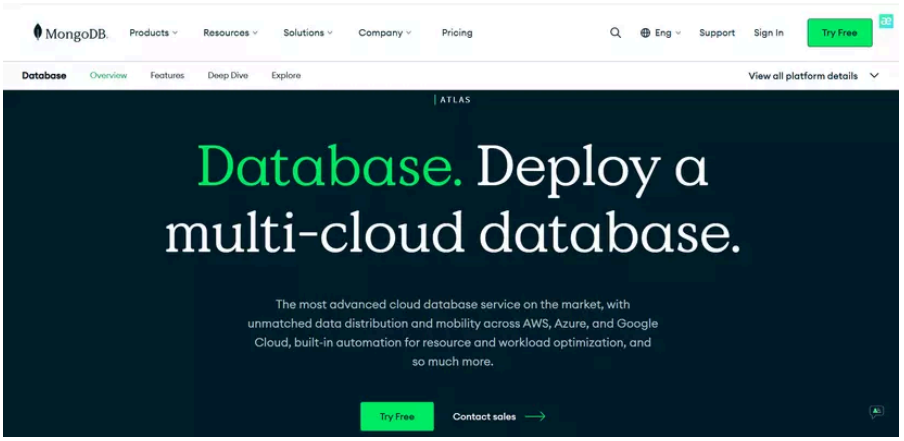
Hình 6: Minh họa cho mô hình Sharding

2.2.2.2. Hạ tầng triển khai

2.2.2.2.1. Sử dụng Cloud (MongoDB Atlas):

MongoDB Atlas là nền tảng cơ sở dữ liệu đám mây do MongoDB cung cấp, giúp người dùng thiết lập và quản lý cơ sở dữ liệu một cách nhanh chóng mà không

cần phải lo lắng về quản lý thủ công. Người dùng sẽ được cấp 512 MB miễn phí, rất phù hợp để bắt đầu và thử nghiệm các tính năng của MongoDB. Atlas hỗ trợ khả năng mở rộng dễ dàng, với các cụm (clusters) được quản lý trên các nhà cung cấp dịch vụ đám mây hàng đầu như AWS, Google Cloud, và Microsoft Azure. Để sử dụng, người dùng chỉ cần truy cập MongoDB Atlas và đăng ký một tài khoản miễn phí. Sau khi đăng ký, có thể tạo một cluster miễn phí, thiết lập quyền truy cập và bắt đầu sử dụng ngay lập tức. Một số ưu điểm nổi bật của MongoDB Atlas là không cần đầu tư vào phần cứng, dễ dàng tích hợp với các ứng dụng hiện có, và luôn được cập nhật tự động với các tính năng mới.

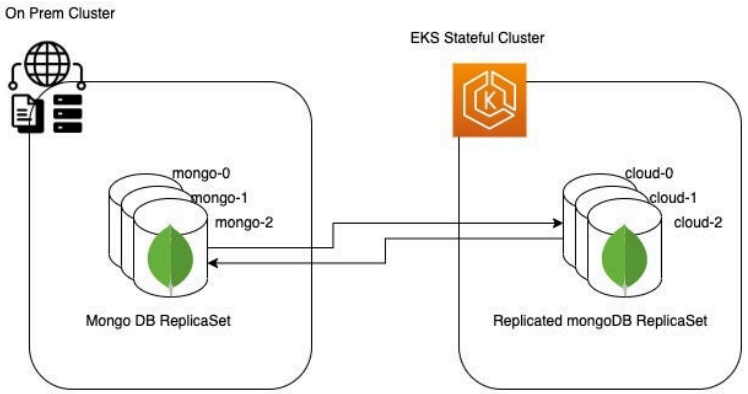


Hình 7: Trang chủ MongoDB Atlas

2.2.2.2.2. Triển khai On-premise

Đây là hình thức triển khai MongoDB trên cơ sở hạ tầng vật lý hoặc máy chủ riêng của tổ chức, phù hợp với các tổ chức có yêu cầu bảo mật cao hoặc muốn kiểm soát hoàn toàn dữ liệu của mình. **Ưu điểm** của hình thức này bao gồm việc kiểm soát hoàn toàn việc quản lý và bảo mật cơ sở dữ liệu, không phụ thuộc vào nhà cung cấp dịch vụ đám mây, và đặc biệt phù hợp cho các tổ chức có cơ sở hạ tầng sẵn có hoặc yêu cầu hoạt động độc lập. Tuy nhiên, nhược điểm là yêu cầu đội ngũ kỹ thuật có chuyên môn để cài đặt, cấu hình và quản lý, cùng với việc cần đầu tư phần cứng, phần mềm và nguồn lực bảo trì hệ thống.

Để triển khai cơ bản, trước tiên hãy tải xuống phiên bản MongoDB Community từ trang chủ MongoDB. Sau đó, tiến hành cài đặt trên hệ điều hành phù hợp như Windows, macOS, hoặc Linux. Bạn có thể tự cấu hình thêm các tệp mongod.conf để điều chỉnh các tham số như cổng (port), lưu trữ dữ liệu, và kết nối mạng. Cuối cùng, khởi động dịch vụ MongoDB và sử dụng MongoDB Shell (mongosh) để thao tác với cơ sở dữ liệu. Việc triển khai này yêu cầu sự chuẩn bị kỹ lưỡng và đội ngũ có kinh nghiệm để đảm bảo hệ thống hoạt động hiệu quả và an toàn



Hình 8: Triển khai on-premise trên MongoDB

2.2.3. Kiến trúc hạ tầng

MongoDB là một cơ sở dữ liệu NoSQL, sử dụng mô hình tài liệu để lưu trữ dữ liệu dưới dạng các document JSON. Kiến trúc của MongoDB bao gồm:

+ **Client:** Người dùng hoặc ứng dụng gửi yêu cầu đến MongoDB thông qua driver.
+ **Shard (nếu có):** Trong trường hợp dữ liệu lớn, MongoDB hỗ trợ phân mảnh (sharding) để phân phối dữ liệu trên nhiều server, tăng tính mở rộng.
+ **Config Servers:** Lưu trữ thông tin về cấu hình và vị trí của các shard trong cluster.
+ **MongoDB Instances:** Các phiên bản MongoDB thực tế chứa dữ liệu. Mỗi instance có thể là một server độc lập hoặc một phần của cluster.
+ **Replica Sets:** Các phiên bản này có thể được cấu hình thành các replica set để cung cấp tính khả dụng cao và bảo vệ dữ liệu. Một replica set có một primary node và nhiều secondary nodes.
+ **Storage Layer:** Chịu trách nhiệm lưu trữ dữ liệu trên đĩa. MongoDB sử dụng WiredTiger hoặc MMAPv1 để quản lý dữ liệu.
+ **Query Engine:** Xử lý các truy vấn và thao tác dữ liệu, cung cấp khả năng tìm kiếm và phân tích dữ liệu.

2.2.4. Làm việc với dữ liệu trong MongoDB

Hệ quản trị CSDL MongoDB lưu trữ dữ liệu theo cấu trúc phân cấp:

database > collection > document > field:value

Document tuân thủ **định dạng BSON (gần giống JSON)**, một document gồm một hay nhiều cặp field:value như đã trình bày ở trên.

2.2.4.1. Kết nối đến MongoDB

Để bắt đầu làm việc với MongoDB, trước tiên, bạn cần kết nối đến cơ sở dữ liệu. Có thể sử dụng MongoDB Shell hoặc các thư viện của ngôn ngữ lập trình như Node.js, Python, Java, … Và cụ thể trong báo cáo, thì nhóm sẽ kết nối đến thư viện của Python để tiện cho việc xử lý dữ liệu.

Tạo CSDL và bộ sưu tập

+ **Tạo cơ sở dữ liệu:** Sử dụng lệnh use <tên_cơ_sở_dữ_liệu> để tạo hoặc chuyển đến cơ sở dữ liệu.
+ **Tạo bộ sưu tập:** Sử dụng lệnh db.createCollection("<tên_bộ_sưu_tập>").

Thêm dữ liệu

+ Để thêm dữ liệu vào bộ sưu tập, có thể dùng lệnh insertOne() hoặc insertMany().

Truy vấn dữ liệu

+ MongoDB có khả năng truy vấn mạnh mẽ với cú pháp đơn giản. Đó là dùng lệnh find để truy vấn.

Cập nhật và xóa dữ liệu

+ **Cập nhật:** Sử dụng lệnh updateOne() hoặc updateMany().
+ **Xóa:** Sử dụng deleteOne() hoặc deleteMany().

2.2.5. Cách tối ưu MongoDB

MongoDB là một hệ quản trị cơ sở dữ liệu NoSQL phổ biến, nổi bật với tính linh hoạt và khả năng mở rộng. Tuy nhiên, để đảm bảo hiệu suất tối ưu khi làm việc với MongoDB, thì cần áp dụng một số chiến lược cụ thể nhằm nâng cao hiệu quả.

+ **Kiến trúc dữ liệu hợp lý:** Một trong những yếu tố quan trọng nhất trong tối ưu hóa MongoDB là thiết kế kiến trúc dữ liệu sao cho hợp lý. Cần xác định rõ mô hình dữ liệu và cách thức tổ chức các tài liệu. Sử dụng các mảng và nhúng tài liệu khi cần thiết để giảm số lượng truy vấn. Ví dụ, nếu có nhiều đối tượng liên quan, có thể nhúng chúng vào tài liệu cha thay vì sử dụng liên kết nhiều tài liệu. Điều này giúp giảm độ trễ và tăng tốc độ truy xuất dữ liệu.
+ **Chỉ số (Indexing):** Chỉ số là một công cụ mạnh mẽ để tối ưu hóa tìm kiếm trong MongoDB. Việc tạo chỉ số cho các trường thường xuyên được truy vấn sẽ giúp cải thiện đáng kể hiệu suất truy vấn. Tuy nhiên, cần cân nhắc số lượng chỉ số vì quá nhiều chỉ số có thể làm chậm quá trình ghi dữ liệu. Tối ưu hóa các chỉ số bằng cách sử dụng chỉ số hợp

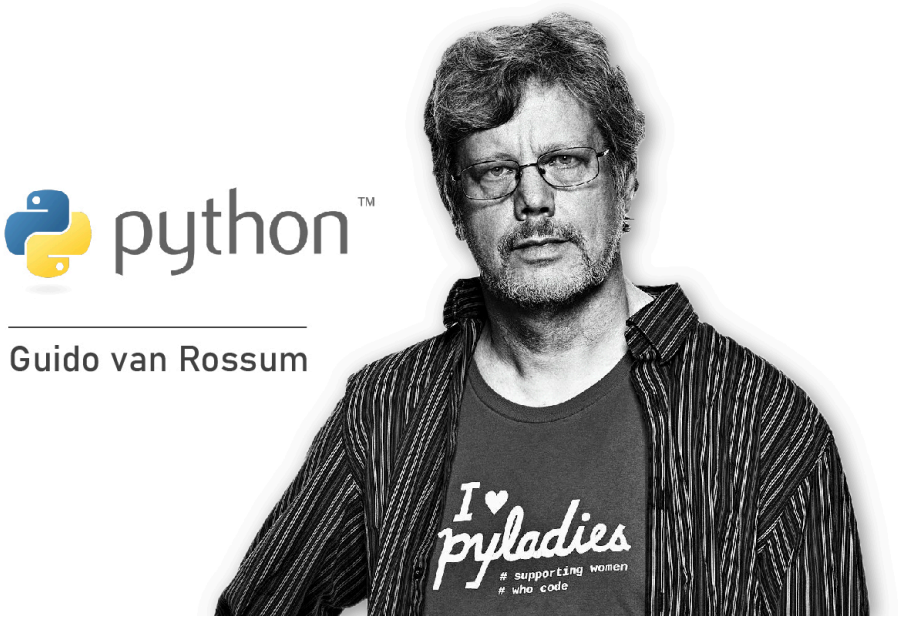
nhất (compound indexes) cho các trường thường xuyên truy vấn cùng nhau.

+ **Cấu hình shard (Sharding):** Đối với các dự án lớn với lượng dữ liệu khổng lồ, sharding là một giải pháp cần thiết. Sharding cho phép phân tán dữ liệu trên nhiều máy chủ, giúp giảm tải cho cơ sở dữ liệu và tăng khả năng mở rộng. Khi cấu hình sharding, việc lựa chọn shard key phù hợp là rất quan trọng để đảm bảo dữ liệu được phân phối đồng đều và tránh tình trạng tắc nghẽn.
+ **Tinh chỉnh cấu hình:** MongoDB cung cấp nhiều tùy chọn cấu hình cho phép người dùng điều chỉnh hoạt động của cơ sở dữ liệu. Việc tùy chỉnh các tham số như kích thước bộ nhớ đệm (cache size), số lượng kết nối tối đa và thời gian timeout sẽ giúp cải thiện hiệu suất. Đặc biệt, cần theo dõi sử dụng tài nguyên để có điều chỉnh kịp thời.
+ **Giám sát và tối ưu hóa hiệu suất:** Cuối cùng, việc theo dõi hiệu suất MongoDB thường xuyên là rất cần thiết. Sử dụng các công cụ giám sát như MongoDB Atlas hoặc các phần mềm bên thứ ba để theo dõi tình trạng hoạt động của cơ sở dữ liệu. Qua đó, người dùng có thể xác định các nút cổ chai trong hệ thống và điều chỉnh cấu hình, tối ưu hóa truy vấn hoặc thêm phần cứng nếu cần thiết.

2.3. Ngôn ngữ lập trình Python

2.3.1. Lịch sử hình thành

Python là ngôn ngữ lập trình tập trung vào sự đơn giản và dễ sử dụng hướng đối tượng, cấp cao, mạnh mẽ. Ngày nay, Pthon được sử dụng rộng rãi trong nhiều lĩnh vực và là một trong những ngôn ngữ lập trình được sử dụng nhiều nhất trên thế giới. Được phát triển bởi Guido van Rossum vào năm 1991, Python nhanh chóng trở thành một lựa chọn hàng đầu trong cộng đồng lập trình nhờ vào cú pháp đơn giản, dễ đọc và dễ học. Một trong những điểm mạnh nổi bật của Python là cú pháp rõ ràng, giúp lập trình viên viết mã hiệu quả mà không cần quá nhiều ký tự đặc biệt như trong các ngôn ngữ lập trình khác.



Hình 9: Lịch sử hình thành Python

2.3.2. Ứng dụng của Python

Python là một trong những ngôn ngữ lập trình phổ biến nhất hiện nay, được ứng dụng rộng rãi trong nhiều lĩnh vực thực tế nhờ cú pháp đơn giản, thư viện phong phú và khả năng mở rộng mạnh mẽ.



Hình 10: Ứng dụng thực tế của Python

Trong lĩnh vực trí tuệ nhân tạo (AI) và học máy (Machine Learning), Python là lựa chọn hàng đầu với các thư viện mạnh mẽ như TensorFlow, PyTorch và Scikit-learn. Nó giúp các nhà khoa học dữ liệu xây dựng và triển khai các mô hình AI hiệu quả.

Python cũng đóng vai trò quan trọng trong **phát triển web**, với các framework như Django và Flask giúp lập trình viên tạo ra các ứng dụng web linh hoạt và bảo mật cao.

Ngoài ra, Python được sử dụng nhiều trong **tự động hóa và phát triển phần mềm**, giúp đơn giản hóa các tác vụ như xử lý dữ liệu, kiểm thử phần mềm và quản lý hệ thống. **Trong tài chính và phân tích dữ liệu**, Python hỗ trợ xử lý dữ liệu lớn, tạo báo cáo và mô phỏng các mô hình tài chính thông qua Pandas và NumPy.

2.3.3. Ưu điểm và nhược điểm của Python

Về ưu điểm, đây là một ngôn ngữ có hình thức sáng sủa, cấu trúc roc ràng, cú pháp ngắn gọn nên dễ học – dễ làm – dễ cài đặt. Python có trên tất cả các nền tảng hệ điều hành từ UNIX, MS – DOS, Mac OS, Windows và Linux và các OS khác thuộc họ Unix. Ngôn ngữ này tương thích mạnh mẽ với Unix, hardware, third-party software với số lượng thư viện khổng lồ (400 triệu người sử dụng). Nhiều thư viện có sẵn ví dụ thư viện data mining Scikit-learn , Pandas. Python với tốc độ xử lý cực nhanh, python có thể tạo ra những chương trình từ những script siêu nhỏ tới những phần mềm cực lớn như Blender 3D.

Nhược điểm tồn tại của ngôn ngữ này gồm việc Python không có các thuộc tính như :protected,private hay public, không có vòng lặp do…while và switch….case. Và Python mặc dù nhanh hơn so với PHP, nhưng lại không nhanh hơn so với C++, Java.

2.4. Công cụ xử lý Big Data PySpark

2.4.1. PySpark là gì?

PySpark là một giao diện lập trình ứng dụng (API) Python cho Apache Spark, một công cụ mạnh mẽ giúp thực hiện các tác vụ xử lý big data và thời gian thực trong một môi trường phân tán, sử dụng ngôn ngữ lập trình Python. Điều đặc biệt của PySpark không chỉ là khả năng xử lý dữ liệu mạnh mẽ, mà còn là sự cung cấp của shell PySpark, giúp người dùng có thể phân tích dữ liệu một cách tương tác, linh hoạt và trực quan. PySpark kết hợp khả năng sử dụng dễ dàng của Python với sức mạnh của Apache Spark, nhằm mục đích xử lý và phân tích dữ liệu ở mọi quy mô cho mọi người, đặc biệt là những người quen thuộc với Python. Ngoài ra, PySpark cung cấp đầy đủ hỗ trợ cho tất cả các tính năng chính của Spark, bao gồm Spark SQL cho truy vấn dữ liệu dạng SQL, DataFrames cho xử lý dữ liệu cấu trúc, Structured Streaming cho xử lý dữ liệu luồng cấu trúc, Machine Learning cho các ứng dụng, và Spark Core cho các tác vụ xử lý dữ liệu cơ bản.



Hình 11: Công cụ PySpark

2.4.2. Ứng dụng thực tế của PySpark

PySpark là một trong những công cụ phổ biến nhất trong lĩnh vực Data Science và Machine Learning, vì nó sử dụng Python – ngôn ngữ lập trình có nhiều thư viện hỗ trợ cho khoa học dữ liệu như NumPy và TensorFlow. PySpark giúp bạn xây dựng các ứng dụng học máy hiệu quả và mạnh mẽ trên dữ liệu khổng lồ, có thể lên đến hàng tỷ hay cả nghìn tỷ bản ghi, với tốc độ cao gấp 100 lần so với các ứng dụng Python bình thường. Nhiều công ty lớn như Amazon, Walmart, Trivago, Sanofi, và Runtastic đã áp dụng PySpark vào hoạt động của mình. PySpark còn được sử dụng trong đa dạng các ngành nghề khác nhau: Y tế, tài chính, giáo dục, giải trí, dịch vụ tiện ích, thương mại điện tử và nhiều lĩnh vực khác.



Hình 12: Ứng dụng của PySpark

2.4.3. Ưu điểm và nhược điểm của PySpark

Về ưu điểm, PySpark cung cấp một API Python quen thuộc, giúp người dùng làm việc với Spark một cách dễ dàng. Chúng hỗ trợ nhiều nguồn và định dạng dữ liệu khác nhau, bao gồm dữ liệu có cấu trúc, bán cấu trúc và không có cấu trúc. Tích hợp tốt với các thư viện và công cụ Python phổ biến khác, như NumPy, pandas và scikit-learn.

Cũng như có khả năng mở rộng các công việc Spark trên một cụm máy tính, giúp tăng hiệu suất và xử lý được lượng dữ liệu lớn hơn.

Tuy nhiên, PySpark vẫn tồn tại các nhược điểm liên quan tới hiệu suất và các công cụ liên quan. PySpark có thể gây ra một số độ trễ do quá trình chuyển đổi giữa Python và JVM (Java Virtual Machine). Điều này làm cho PySpark chạy chậm hơn so với việc sử dụng Spark trực tiếp bằng Scala. Mặc dù Spark có khả năng tối ưu hóa việc sử dụng bộ nhớ, nhưng khi chạy PySpark, người dùng có thể gặp vấn đề về quản lý bộ nhớ do cách Python xử lý đối tượng và rác thải bộ nhớ (Garbage Collection). Điều này có thể gây ra lỗi OutOfMemoryError khi xử lý dữ liệu lớn. Hỗ trợ hạn chế cho một số thư viện Python nên PySpark không thể tận dụng toàn bộ sức mạnh của các thư viện Python phổ biến như Pandas hay NumPy trong môi trường phân tán, vì chúng không được thiết kế để chạy song song trên nhiều nút trong Spark Cluster. Cấu hình và Debugging phức tạp chủ yếu vì PySpark yêu cầu cấu hình chính xác để đạt hiệu suất tối ưu. Việc tìm lỗi (debugging) cũng khó khăn hơn vì lỗi thường xuất phát từ Spark JVM thay vì từ Python, dẫn đến thông báo lỗi khó hiểu và khó xử lý. Không phù hợp cho xử lý dữ liệu nhỏ do PySpark được thiết kế để xử lý dữ liệu lớn trên nhiều máy chủ. Nếu dữ liệu nhỏ, việc sử dụng PySpark có thể gây lãng phí tài nguyên, trong khi các công cụ như Pandas hoặc Dask sẽ phù hợp hơn.

2.5. Mô tả bộ dữ liệu

2.5.1. Giới thiệu bộ dữ liệu

Bộ dữ liệu bao gồm 165474 quan sát cung cấp thông tin về clickstream từ một cửa hàng trực tuyến chuyên cung cấp quần áo dành cho phụ nữ mang thai. Dữ liệu được thu thập trong vòng năm tháng của năm 2008 và bao gồm nhiều thuộc tính liên quan đến hành vi duyệt web của người dùng. Bộ dữ liệu bao gồm 14 biến mô tả chi tiết về từng lần truy cập của khách hàng, giúp phân tích hành vi người dùng và khám phá các mô hình mua sắm trực tuyến. Mỗi dòng dữ liệu tương ứng với một thao tác trong một phiên truy cập, giúp theo dõi trình tự duyệt web của khách hàng từ lúc vào trang đến khi rời đi. Nguồn dữ liệu: Clickstream Data for Online Shopping**.**

Các biến có trên dữ liệu bao gồm:

Bảng 2.1. Giới thiệu các biến có trong tập dữ liệu

BIẾN	MÔ TẢ	Ý NGHĨA
YEAR	Year of data collection (2008)	Năm thu thập dữ liệu (2008)
MONTH	Month of the year (from April (4) to August (8))	Tháng trong năm (từ 4 đến 8)
DAY	Day of the month	Ngày trong tháng
ORDER	Sequence of clicks during	Thứ tự các lượt nhấp

	one session	trong một phiên
COUNTRY	Country of the IP address	Quốc gia của địa chỉ IP truy cập
SESSION ID	Session identifier	Mã định danh của phiên truy cập
PAGE 1	Main product category	Danh mục chính của sản phẩm
PAGE 2	Specific product code (217 products)	Mã sản phẩm cụ thể (217 sản phẩm)
COLOUR	Product color	Màu sắc của sản phẩm
LOCATION	Photo location on the webpage	Vị trí hiển thị ảnh trên trang web
MODEL PHOTOGRAPHY	Product image type (en face or profile)	Kiểu ảnh sản phẩm (chính diện hoặc góc nghiêng)
PRICE	Product price in US dollars	Giá sản phẩm (USD)
PRICE 2	Price above category average? (1: Yes, 2: No)	Giá có cao hơn mức trung bình danh mục không (1: Có, 2: Không)
PAGE	Page number within the website (1 to 5)	Số trang trong website

Dựa vào bảng thông tin các thuộc tính trên, có thể thấy rằng có nhiều yếu tố khác nhau ảnh hưởng đến hành vi duyệt web và mua sắm của người dùng trong cửa hàng trực tuyến. Điều này cho phép chúng ta xây dựng một hệ thống đề xuất sản phẩm phù hợp dựa trên dữ liệu clickstream. Một số giả định ban đầu có thể đưa ra như sau:

- **PAGE 1 (Danh mục chính):** Người dùng có xu hướng xem các danh mục sản phẩm yêu thích, ví dụ như quần, váy hay áo blouse. Việc theo dõi danh mục được truy cập nhiều nhất giúp đề xuất các sản phẩm liên quan.
- **COLOUR (Màu sắc sản phẩm):** Nếu người dùng thường xuyên xem sản phẩm có màu cụ thể (ví dụ: đen, trắng), hệ thống có thể đề xuất các mặt hàng tương tự về màu sắc.
- **LOCATION (Vị trí ảnh trên trang web):** Sản phẩm được hiển thị ở những vị trí thu hút có thể ảnh hưởng đến quyết định mua hàng. Nếu một nhóm khách hàng thường xuyên nhấp vào sản phẩm ở vị trí “top left”, ta có thể tối ưu giao diện đề xuất dựa trên hành vi này.

- **COUNTRY (Quốc gia truy cập):** Xu hướng mua sắm có thể khác nhau theo từng khu vực. Người dùng từ các quốc gia khác nhau có thể quan tâm đến các loại sản phẩm hoặc mức giá khác nhau.
- **PRICE (Giá sản phẩm):** Nếu một người dùng chủ yếu quan tâm đến sản phẩm trong một khoảng giá nhất định, hệ thống có thể ưu tiên đề xuất các sản phẩm phù hợp với ngân sách của họ.

Dựa trên những yếu tố trên, một số câu hỏi có thể được đặt ra:

- *Có thể dự đoán sản phẩm mà khách hàng có thể quan tâm tiếp theo không?*
- *Yếu tố nào quan trọng nhất trong việc xác định sản phẩm phù hợp với sở thích người dùng?*
- *Làm thế nào để sử dụng Hadoop và MapReduce để xử lý lượng lớn dữ liệu clickstream nhằm cải thiện hệ thống đề xuất sản phẩm?*

2.5.2. Tại sao cần sử dụng Big Data để giải quyết bài toán

Dựa vào bối cảnh và mục tiêu của bài toán, ta cần phân tích và khai thác dữ liệu clickstream của người dùng trên website thương mại điện tử để đề xuất sản phẩm phù hợp. Việc này đòi hỏi xử lý một lượng dữ liệu lớn, có cấu trúc phức tạp, bao gồm trình tự click, danh mục sản phẩm, màu sắc, vị trí hiển thị trên trang web, giá sản phẩm, quốc gia của người dùng, v.v. Để tìm ra các quy luật mua sắm và đưa ra đề xuất hiệu quả, chúng ta cần một phương pháp có thể xử lý dữ liệu lớn một cách nhanh chóng và tối ưu.

Hadoop và MapReduce là công nghệ phù hợp để giải quyết bài toán này vì **Hadoop Distributed File System (HDFS)** giúp lưu trữ và quản lý dữ liệu lớn hiệu quả, cho phép xử lý dữ liệu phân tán trên nhiều máy chủ. Trong khi đó **MapReduce** giúp xử lý và trích xuất thông tin từ clickstream theo cách phân tán, giúp tổng hợp dữ liệu từ nhiều nguồn và tìm ra các xu hướng mua sắm mà không cần xử lý trên một máy chủ duy nhất.

Như vậy, việc sử dụng Hadoop giúp giảm chi phí và tối ưu hiệu suất so với các hệ thống truyền thống khi xử lý dữ liệu lớn trong thương mại điện tử.

Để giải quyết bài toán đề xuất sản phẩm, ta có thể sử dụng MapReduce để phân tích tần suất click, chuỗi hành vi của người dùng và danh mục sản phẩm được quan tâm. Sau khi dữ liệu được tổng hợp, hệ thống có thể xác định các sản phẩm thường xuyên được mua cùng nhau hoặc các sản phẩm mà người dùng có xu hướng click tiếp theo.

Sau khi xác định được các sản phẩm liên quan, ta có thể xây dựng hệ thống đề xuất dựa trên:

- **User-based filtering**: Đề xuất sản phẩm dựa trên hành vi của những người dùng có hành vi mua sắm tương tự.
- **Item-based filtering**: Gợi ý sản phẩm dựa trên các sản phẩm có đặc điểm tương đồng với sản phẩm mà người dùng đang quan tâm.
- **Rule-based recommendations**: Sử dụng luật kết hợp để tìm ra các nhóm sản phẩm thường xuyên được xem cùng nhau.

Với các phương pháp trên, hệ thống có thể đưa ra gợi ý chính xác hơn, từ đó tăng tỷ lệ chuyển đổi, cải thiện trải nghiệm người dùng và tối ưu hóa doanh thu cho doanh nghiệp.

Vì những lý do trên, nhóm xây dựng bài toán: Sử dụng Công nghệ (Hadoop, MapReduce) kết hợp với nền tảng Big Data trong việc phân tích dữ liệu clickstream và đề xuất sản phẩm phù hợp cho người dùng trên website thương mại điện tử.

CHƯƠNG 3: MÔ TẢ THUẬT TOÁN, DEMO ỨNG DỤNG

3.1. Mô tả thuật toán

3.1.1. Thu thập và lưu trữ dữ liệu

Bước thu thập dữ liệu Clickstream bắt đầu với việc lựa chọn tập dữ liệu phù hợp từ Kaggle, một nền tảng chia sẻ dữ liệu phổ biến, nơi các tập dữ liệu thường có sẵn dưới dạng tệp CSV. Sau khi xác định được tập dữ liệu mong muốn, tệp CSV được tải xuống và kiểm tra sơ bộ nhằm đảm bảo tính toàn vẹn và phù hợp của dữ liệu. Quá trình kiểm tra này bao gồm việc xác định cấu trúc dữ liệu, số lượng dòng và cột, kiểm tra các giá trị bị thiếu hoặc dữ liệu không hợp lệ. Cụ thể, tập dữ liệu Clickstream thường chứa các trường quan trọng như ID người dùng, thời gian truy cập, địa chỉ trang web đã xem, loại sự kiện (ví dụ: nhấp chuột, thêm vào giỏ hàng, hoàn tất thanh toán), cũng như các thông tin bổ sung như thiết bị sử dụng, vị trí địa lý và nguồn truy cập. Việc kiểm tra và tiền xử lý này rất quan trọng nhằm đảm bảo dữ liệu đầu vào có chất lượng cao, tránh gây sai lệch trong quá trình phân tích sau này.

```
from pymongo import MongoClient

# Kết nối với MongoDB
client = MongoClient("mongodb://localhost:27017/")

# Chọn database và collection
db = client["BI"]
collection = db["Clickstream"]
```

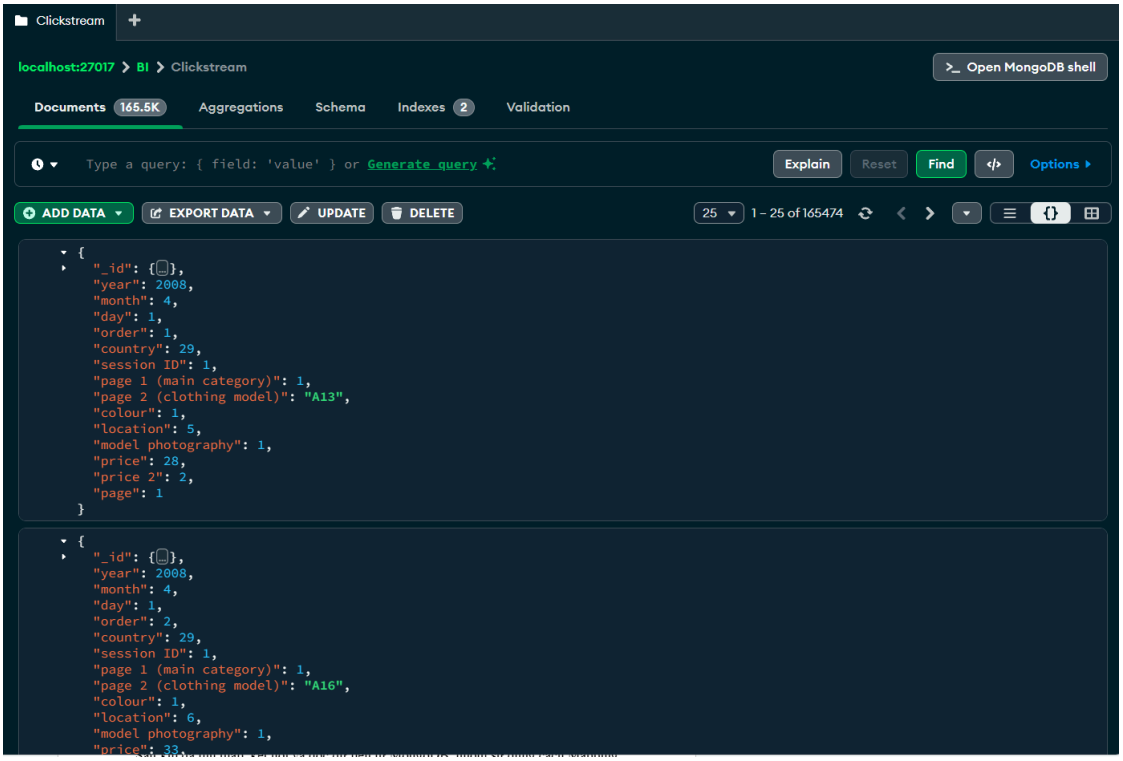
```
#Hàm gọi collection
def get_collection():
    return collection

# Kiểm tra kết nối bằng cách lấy một bản ghi
for record in collection.find().limit(5):
    print(record)
```

Sau khi dữ liệu CSV được xác nhận là hợp lệ, bước tiếp theo là chuyển đổi nó sang định dạng JSON để phù hợp với hệ quản trị cơ sở dữ liệu MongoDB, vốn được thiết kế để làm việc với dữ liệu có cấu trúc linh hoạt và không ràng buộc theo mô hình bảng như SQL. Quá trình chuyển đổi được thực hiện bằng cách sử dụng các thư viện Python như pandas để đọc dữ liệu từ CSV và json để xuất ra định dạng JSON. Tiếp theo, để lưu trữ dữ liệu vào MongoDB, Python được sử dụng kết hợp với thư viện **PyMongo**, giúp thiết lập kết nối giữa ứng dụng Python và cơ sở dữ liệu MongoDB. Sau đó, nhóm thực hiện kết nối và lấy dữ liệu từ MongoDB bằng cách sử dụng **Connector.get_collection()** để kết nối với MongoDB và **lấy dữ liệu từ collection**. Sau đó, dữ liệu được **chuyển đổi thành DataFrame** bằng **pd.DataFrame(data)** để dễ dàng thao tác và phân tích.

	A	B	C	D	E	F	G	H	I	J	K	L	M	N
1	year	month	day	order	country	session ID	page 1 (ma	page 2 (clo	colour	location	model pho	price	price 2	page
2	2008	4	1	1	29	1	1	A13	1	5	1	28	2	1
3	2008	4	1	2	29	1	1	A16	1	6	1	33	2	1
4	2008	4	1	3	29	1	2	B4	10	2	1	52	1	1
5	2008	4	1	4	29	1	2	B17	6	6	2	38	2	1
6	2008	4	1	5	29	1	2	B8	4	3	2	52	1	1
7	2008	4	1	6	29	1	3	C56	6	1	2	57	1	4
8	2008	4	1	7	29	1	3	C57	5	1	2	33	2	4
9	2008	4	1	8	29	1	4	P67	9	5	1	38	1	4
10	2008	4	1	9	29	1	4	P82	6	4	2	48	1	5
11	2008	4	1	1	29	2	2	B31	9	5	1	57	1	2
12	2008	4	1	2	29	2	2	B21	12	1	1	67	1	2
13	2008	4	1	3	29	2	2	B24	11	2	1	57	1	2
14	2008	4	1	4	29	2	2	B27	2	3	1	57	1	2
15	2008	4	1	5	29	2	1	A10	3	4	1	38	2	1
16	2008	4	1	6	29	2	1	A10	3	4	1	38	2	1
17	2008	4	1	7	29	2	2	B27	2	3	1	57	1	2
18	2008	4	1	8	29	2	4	P1	3	1	1	38	1	1
19	2008	4	1	9	29	2	4	P34	9	6	2	48	1	2
20	2008	4	1	10	29	2	4	P33	9	5	1	43	1	2
21	2008	4	1	1	21	3	2	B17	6	6	2	38	2	1
22	2008	4	1	2	21	3	3	C4	4	2	1	48	1	1
23	2008	4	1	3	21	3	3	C7	13	3	1	48	1	1
24	2008	4	1	4	21	3	3	C10	9	4	2	28	2	1
25	2008	4	1	5	21	3	3	C17	14	6	1	48	1	1
26	2008	4	1	6	21	3	4	P77	7	2	1	43	1	5
27	2008	4	1	1	21	4	1	A34	2	6	1	38	2	2

Hình 13: Dữ liệu được lưu trữ dưới dạng CSV



Hình 14: Dữ liệu lưu trữ dưới dạng JSON trong MongoDB

3.1.2. Tiền xử lý dữ liệu

Sau khi đã thu thập, kết nối và đọc dữ liệu từ MongoDB, nhóm sử dụng cách Mapping các giá trị tại để các cột **country, page 1 (main category), location, colour, model photography, month** và **price_category** được chuyển đổi từ các giá trị số sang các giá trị văn bản dễ hiểu hơn. Các cột được chuyển đổi sang kiểu dữ liệu phù hợp, ví dụ **country** được chuyển sang **kiểu string.** Ngoài ra nhóm cũng sử dụng câu lệnh **df.columns.str.strip()** nhằm loại bỏ khoảng trắng thừa ở đầu và cuối tên cột (nếu có). Điều này giúp tránh lỗi khi truy xuất cột có tên không đồng nhất.

```
df = pd.DataFrame(data)
df.columns = df.columns.str.strip()  # Clean column names

# Country Mapping
country_map = {
      "1": "Australia", "2": "Austria", "3": "Belgium", "4": "British
Virgin Islands",
      "5": "Cayman Islands", "6": "Christmas Island", '7': "Croatia",
'8': "Cyprus",
      '9': "Czech Republic", '10': "Denmark", "11": "Estonia", "12":
"unidentified",
      "13": "Faroe Islands", "14": "Finland", "15": "France", "16":
"Germany",
```

```
    '17': "Greece", "18": "Hungary", "19": "Iceland", "20": "India",
    "21": "Ireland", "22": "Italy", "23": "Latvia", "24": "Lithuania",
      "25": "Luxembourg", "26": "Mexico", "27": "Netherlands", "28":
"Norway",
    "29": "Poland", "30": "Portugal", "31": "Romania", '32': "Russia",
      '33': "San Marion", "34": "Slovakia", "35": "Slovenia", "36":
"Spain",
    '37': "Sweden", "38": "Switzerland", "39": "Ukraine", "40": "United
Arab Emirates",
    "41": "United Kingdom", "42": "USA", "43": "biz (.biz)", "44": "com
(.com)",
    "45": "int (.int)", "46": "net (.net)", "47": "org (*.org)"
}
df["country"] = df["country"].astype("string").map(country_map)

# Product Category Mapping
page1_map = {1: "trousers", 2: "skirts", 3: "blouses", 4: "sale"}
df["page 1 (main category)"] = df["page 1 (main
category)"].map(page1_map)

# Location Mapping
location_map = {
    1: "top left", 2: "top in the middle", 3: "top right",
    4: "bottom left", 5: "bottom in the middle", 6: "bottom right"
}
df["location"] = df["location"].map(location_map)

# Color Mapping
color_map = {
    1: "beige", 2: "black", 3: "blue", 4: "brown", 5: "burgundy",
    6: "gray", 7: "green", 8: "navy blue", 9: "of many colors",
    10: "olive", 11: "pink", 12: "red", 13: "violet", 14: "white"
}
df["colour"] = df["colour"].map(color_map)

# Model Photography Mapping
model_map = {1: "En face", 2: "Profile"}
df["model photography"] = df["model photography"].map(model_map)

# Month Mapping
month_map = {4: "April", 5: "May", 6: "June", 7: "July", 8: "August"}
df["month"] = df["month"].map(month_map)
```

```
# Price Category
df["price_category"] = df["price 2"].map({
    1: "above average", 2: "below average"
}).astype("category")
```

3.1.3. Trực quan hóa dữ liệu và đưa lên Streamlit

Dữ liệu tiếp tục được xử lý bằng cách chuyển đổi các mã số thành thông tin dễ hiểu, chẳng hạn như **tên quốc gia, danh mục sản phẩm, vị trí hiển thị trên trang web, màu sắc sản phẩm và các thuộc tính khác**. Các thao tác quan trọng bao gồm **nhóm dữ liệu (group by), tính tổng (sum), đếm số lượng duy nhất (nunique) và tạo bảng pivot để hỗ trợ phâ**n tích chuyên sâu. Mã nguồn tập trung vào việc phân tích hành vi khách hàng, chẳng hạn như theo dõi số lượt truy cập theo quốc gia, khung thời gian, danh mục sản phẩm và mức giá. Một số phép tính phổ biến được thực hiện bao gồm:

+ Tổng số phiên truy cập duy nhất.
+ Số lần nhấp chuột trung bình mỗi phiên.
+ So sánh mức độ quan tâm giữa các nhóm sản phẩm dựa trên giá cả.

Để làm nổi bật xu hướng trong dữ liệu, nhiều phương pháp trực quan hóa được áp dụng, chẳng hạn như:

+ Biểu đồ cột để hiển thị số lượt truy cập theo quốc gia.
+ Biểu đồ scatter thể hiện số lần nhấp chuột trung bình mỗi phiên.
+ Heatmap giúp phân tích mối quan hệ giữa danh mục sản phẩm và mức giá.

Trong quá trình xử lý, các hàm Group By và Aggregation được sử dụng rộng rãi để nhóm dữ liệu theo nhiều tiêu chí khác nhau (quốc gia, tháng, danh mục, v.v.) và thực hiện các phép tính tổng hợp như sum, count, nunique nhằm hỗ trợ việc phân tích và đưa ra các insights hữu ích.

Về trực quan hóa, các hàm px.bar(), px.scatter(), px.pie() được dùng để tạo các biểu đồ, trong khi, hàm sns.heatmap() dùng Seaborn để vẽ heatmap hiển thị mật độ dữ liệu theo danh mục và mức giá. Để so sánh, nhóm dùng các vẽ biểu đồ cột nhóm để so sánh số lượt xem theo danh mục và giá cả và tạo nhiều biểu đồ trong cùng một hình để so sánh dữ liệu.

```
# 1 Product View Analysis by Price Category (Heatmap & Bar Chart)
st.subheader("Product View Analysis by Price Category")
df_price_category = df.groupby(["price", "page 1 (main
category)"]).size().reset_index(name="views")
fig, ax = plt.subplots(figsize=(10, 6))
sns.heatmap(df_price_category.pivot(index="price", columns="page 1
(main category)", values="views"), cmap="coolwarm", ax=ax)
st.pyplot(fig)
```

```
fig, ax = plt.subplots(figsize=(10, 6))
sns.barplot(x=df_price_category["price"], y=df_price_category["views"],
hue=df_price_category["page 1 (main category)"], ax=ax)
st.pyplot(fig)

# 2 Most Viewed Product Colors (Bar Chart)
st.subheader("Most Viewed Product Colors")
color_counts = df["colour"].value_counts().reset_index()
color_counts.columns = ["Color", "Views"]

# Define a color mapping dictionary
color_palette_map = {
    "beige": "#F5F5DC",
    "black": "#000000",
    "blue": "#0000FF",
    "brown": "#A52A2A",
    "burgundy": "#800020",
    "gray": "#808080",
    "green": "#008000",
    "navy blue": "#000080",
    "of many colors": "#808080",
# Neutral gray for mixed colors
    "olive": "#808000",
    "pink": "#FFC0CB",
    "red": "#FF0000",
    "violet": "#EE82EE",
    "white": "#FFFAFA"
}

# Apply the color mapping
bar_colors = color_counts["Color"].map(color_palette_map)

# Convert mapped colors to a list
bar_colors = color_counts["Color"].map(color_palette_map).tolist()

# Corrected Bar Chart with Mapped Colors
fig, ax = plt.subplots(figsize=(10, 6))
sns.barplot(x=color_counts["Color"],   y=color_counts["Views"],   ax=ax,
palette=bar_colors)
st.pyplot(fig)
```

```
# 3 Product Views by Page Number (Bar Chart)
st.subheader("Product Views by Page Number")
page_counts = df["page"].value_counts().reset_index()
page_counts.columns = ["Page", "Views"]
fig, ax = plt.subplots(figsize=(10, 6))
sns.barplot(x=page_counts["Page"], y=page_counts["Views"], ax=ax)
st.pyplot(fig)

# 4 Most Attractive Page Sections & Popular Colors (Bubble Chart)
st.subheader("Most Attractive Page Sections & Popular Colors")
df_section_color = df.groupby(["location",
"colour"]).size().reset_index(name="views")
fig, ax = plt.subplots(figsize=(10, 6))
sns.scatterplot(x=df_section_color["location"],
y=df_section_color["colour"], size=df_section_color["views"],
sizes=(20, 200), ax=ax)
st.pyplot(fig)


# Bar Chart: Website Visitors
fig_bar = px.bar(df_most_visitors, x="country", y="visitor_count",
text_auto=True, color="visitor_count",
            title="Actual number of visits by country")

st.plotly_chart(fig_bar, use_container_width=True)

# 🏷️ Product Views by Location (Heatmap)
st.subheader("🛍️ Product Views by Location")
df_location_views = df.groupby(["location", "page 1 (main
category)"]).size().reset_index(name="view_count")

fig_heatmap = px.density_heatmap(df_location_views, x="page 1 (main
category)", y="location", z="view_count",
                                 color_continuous_scale="blues",
title="Product Views by Location and Category")

st.plotly_chart(fig_heatmap)

# Product Category Views
df_product_views = df.groupby("page 1 (main
category)")[["order"]].sum()
```

```
fig_pie = px.pie(df_product_views, names=df_product_views.index,
values="order",
                 title="Total Product Views per Category", hole=0.4)

st.plotly_chart(fig_pie)

# Heatmap: Price Category Views
df_product_views_by_price = df.groupby(["page 1 (main category)",
"price_category"]).size().reset_index(name="view_count")
df_heatmap_data = df_product_views_by_price.pivot(index="page 1 (main
category)", columns="price_category", values="view_count")

plt.figure(figsize=(10, 5))
sns.heatmap(df_heatmap_data, annot=True, fmt="d", cmap="Blues")
st.pyplot(plt)

# Model Photography Style Views
df_views_by_model = df.groupby(["model photography", "page 1 (main
category)"])["order"].sum().reset_index(name="view_count")
fig_model = px.bar(df_views_by_model, x="model photography",
y="view_count", color="page 1 (main category)",
                   title="Which Model Photography Style Attracts More
Views?")
st.plotly_chart(fig_model)

# Views per Country & Day
df_views_by_country_day = df.groupby(["country",
"day"])[["order"]].sum().reset_index()
fig = px.scatter(df_views_by_country_day,
                 x="country",
                 y="day",
                 color="order",
                 size="order",
                 color_continuous_scale="plasma",
# Use 'plasma' for vibrant colors
                   title="Relationship Between Countries and Viewing
Days")

st.plotly_chart(fig)
```

Đoạn code **st.set_page_config(page_title="Website Analytics Dashboard", layout="wide")** được sử dụng để thiết lập cấu hình cho ứng dụng Streamlit, trong đó **page_title** đặt tiêu đề trang web hiển thị trên trình duyệt, còn **layout="wide"** giúp mở

rộng bố cục trang, tối ưu không gian hiển thị cho dữ liệu và biểu đồ. Lệnh **python -m streamlit run "c:/Users/admin/Desktop/CODE/dashboard.py"** (tùy chỉnh theo đường dẫn truy cập trong máy) được sử dụng trong terminal để **chạy ứng dụng Streamlit**, trong đó streamlit run khởi động ứng dụng từ file Python có đường dẫn cụ thể, giúp hiển thị giao diện phân tích dữ liệu trên trình duyệt thông qua localhost.

```
# Streamlit Page Config
st.set_page_config(page_title="Website Analytics Dashboard", layout="wide")
#python -m streamlit run "c:/Users/admin/Desktop/Data visualizations/dashboard.py"
```

3.1.4. Phân tích dữ liệu và đưa ra đề xuất bằng PySpark

Code sử dụng PySpark để xử lý dữ liệu từ MongoDB và tìm cặp sản phẩm thường xuất hiện cùng nhau trong cùng một session. Bắt đầu với việc **Kết nối MongoDB để lấy dữ liệu từ collection Clickstream**. Sau dữ liệu từ MongoDB được chuyển đổi thành danh sách Python. Tiếp đến, để xử lý dữ liệu bằng PySpark nhóm tiến hành **khởi tạo SparkContext**. Nhóm **áp dụng MapReduce** để trích xuất session ID và sản phẩm, nhóm dữ liệu theo session, tạo cặp sản phẩm từ mỗi session, tính tần suất xuất hiện của từng cặp sản phẩm và lưu kết quả

Trong quá trình này, nhóm sử dụng các thư viện:

- **pymongo:** Kết nối và thao tác với MongoDB.
- **pyspark:** Xử lý dữ liệu với Spark RDD.
- **itertools.combinations:** Tạo danh sách cặp sản phẩm từ danh sách sản phẩm trong mỗi session.

Cụ thể các bước thực hiện phía trên, nhóm kết nối và trích xuất dữ liệu từ MongoDB qua việc lấy dữ liệu từ collection Clickstream và chuyển thành danh sách Python bằng hàm **map_function() để trả về các cặp (session_id, item)**. Dữ liệu được đưa vào Spark RDD để xử lý. **Hàm reduce_function(a, b) đã gộp các danh sách sản phẩm theo session ID**. Cùng với **hàm create_pairs(session_data) tạo các cặp sản phẩm từ danh sách item trong session**. Hàm Reduce để tổng hợp số lần xuất hiện của mỗi cặp sản phẩm. Dữ liệu chạy ra được ghi vào **collection frequency_Spark2** để **phục vụ phân tích và truy vấn sau này**. Để hiển thị kết quả, nhóm dùng **hàm collect() để lấy kết quả từ final RDD rồi in kết quả bằng print.** và lưu kết quả vào MongoDB.

```
import pymongo
from pyspark import SparkContext
from itertools import combinations
```

```
# Kết nối đến MongoDB
client = pymongo.MongoClient("mongodb://localhost:27017")  # Kết nối MongoDB
db = client['BI']  # Thay 'BI' bằng tên database của bạn
collection = db['Clickstream']  # Thay 'Clickstream' bằng tên collection của bạn

# Lấy dữ liệu từ MongoDB và chuyển thành danh sách
cursor = collection.find({})  # Lấy tất cả dữ liệu
data = list(cursor)  # Chuyển dữ liệu thành dạng list

# Khởi tạo SparkContext
sc = SparkContext("local", "MapReduceExample")

# Chuyển dữ liệu MongoDB thành RDD
rdd = sc.parallelize(data)

# Hàm map: Mỗi session sẽ phát sinh các cặp khóa-giá trị (pair) từ các món hàng
def map_function(record):
    # Mỗi session ID sẽ có nhiều món hàng
    session_id = record['session ID']
     item = record['page 2 (clothing model)']  # Lấy món hàng từ 'page 2'
    return (session_id, item)

# Hàm reduce: Nhóm các món hàng theo session ID và tạo cặp từ mỗi session
def reduce_function(a, b):
    return a + b

# Áp dụng Map và Reduce
mapped_rdd = rdd.map(map_function)  # Phát sinh session ID và món hàng
grouped_rdd = mapped_rdd.groupByKey()  # Nhóm theo session ID
```

```
# Hàm tạo cặp từ mỗi session ID
def create_pairs(session_data):
    session_id, items = session_data  # session_id và danh sách các món hàng
    pairs = []

    # Tạo tất cả các cặp từ danh sách các món hàng trong mỗi session
    for pair in combinations(items, 2):
        pairs.append(pair)

    return pairs
# Tính toán cặp món hàng cho tất cả session
final_rdd = grouped_rdd.flatMap(lambda x: create_pairs(x))

# Tính tần suất các cặp món hàng
pair_rdd = final_rdd.map(lambda x: (x, 1)).reduceByKey(lambda a, b: a + b)

# Kiểm tra kết quả thu thập từ final_rdd
result = pair_rdd.collect()  # Lấy kết quả từ RDD
print("Collected result:", result)  # In ra kết quả

# Lưu kết quả vào collection mới trong MongoDB
if result:
      output_collection = db['Frequency_Spark2']   # Tạo collection Frequency để lưu kết quả
     output_data = [{"pair": pair, "count": count} for pair, count in result]
     output_collection.insert_many(output_data)   # Lưu vào collection Frequency
     print("Data has been saved to MongoDB in the 'Frequency_Spark2' collection.")
else:
    print("No pairs generated. Nothing to save.")

# Dừng SparkContext
```

```
sc.stop()
```

Đoạn code tiếp theo sử dụng dữ liệu tần suất cặp sản phẩm từ PySpark để tạo hệ thống gợi ý. Nhóm **kết nối MongoDB** để **lấy dữ liệu từ collection Clickstream và Frequency_Spark2** và lấy danh sách sản phẩm đã xem của từng session từ Clickstream cũng như danh sách cặp sản phẩm phổ biến từ frequency_Spark2. Từ đó **tạo gợi ý sản phẩm dựa trên sản phẩm đã xem và lưu kết quả gợi ý vào MongoDB trong collection Final_Recommendations.**

Trong phần này nhóm dùng các thư viện

- **pymongo:** Kết nối MongoDB.
- **collections.defaultdict:** Hỗ trợ lưu trữ dữ liệu dạng dictionary.

Để lấy danh sách sản phẩm mỗi người dùng đã xem, nhóm đã gộp các sản phẩm theo session ID. Kết quả cho ra **dictionary {session_id: [list sản phẩm đã xem]}.** Sau đó nhóm lấy tất cả cặp sản phẩm từ frequency_Spark2 sao cho đảm bảo mỗi cặp có thứ tự cố định. **Trả về dictionary {(sản phẩm A, sản phẩm B): tần suất}.** Để hoàn thiện tính năng đề xuất sản phẩm, đoạn code tiến hành lọc cặp sản phẩm có liên quan đến sản phẩm đã xem rồi sắp xếp theo tần suất xuất hiện giảm dần. Cuối cùng, chọn sản phẩm có tần suất cao nhất nhưng chưa từng xem để gợi ý. Sau đó lưu danh sách gợi ý vào **Final_Recommendations.**

```
import pymongo


# Kết nối đến MongoDB
client = pymongo.MongoClient("mongodb://localhost:27017")  # Kết nối MongoDB


# Kết nối đến database Clickstream và Frequency
clickstream_db = client['BI']  # Database BI chứa Clickstream
clickstream_collection = clickstream_db['Clickstream']  # Collection Clickstream
frequency_collection = clickstream_db['Frequency_Spark2']  # Collection Frequency
output_collection = clickstream_db['Final_Recommendations']  # Tạo collection để lưu các gợi ý


# Hàm để lấy tất cả sản phẩm người dùng đã xem từ Clickstream (cùng session ID)
def get_all_viewed_products():
```

```
    # Sử dụng aggregate để lấy tất cả sản phẩm mà người dùng đã xem theo session ID
    pipeline = [
        {
            "$group": {
                "_id": "$session ID",  # Nhóm theo session ID
                "viewed_products": {"$push": "$page 2 (clothing model)"}  # Lưu danh sách sản phẩm đã xem trong mỗi session
            }
        }
    ]

    cursor = clickstream_collection.aggregate(pipeline)
    viewed_products_by_session = {doc['_id']: doc['viewed_products'] for doc in cursor}
    return viewed_products_by_session

# Hàm để lấy tất cả cặp sản phẩm từ Frequency collection
def get_all_product_pairs():
    # Sử dụng aggregate để lấy tất cả các cặp sản phẩm từ Frequency collection
    pipeline = [
        {
            "$project": {
                "pair": 1,
                "count": 1  # Chọn các cặp sản phẩm và tần suất của chúng
            }
        }
    ]

    cursor = frequency_collection.aggregate(pipeline)
    product_pairs = []

    # Lưu các cặp sản phẩm với tần suất tương ứng
    for rec in cursor:
```

```
        pair = tuple(sorted(rec["pair"]))  # Đảm bảo cặp sản phẩm không bị đảo ngược
        product_pairs.append((pair, rec["count"]))

    return product_pairs

# Hàm để lấy gợi ý sản phẩm dựa trên sản phẩm đã xem, chỉ gợi ý sản phẩm có tần suất cao nhất
def get_recommendations(viewed_products, all_product_pairs):
    recommendations = {}

    for product in viewed_products:
        # Tạo danh sách các cặp sản phẩm có liên quan đến sản phẩm đã xem
        related_pairs = []

        for (pair, count) in all_product_pairs:
            if product in pair:
                related_pairs.append((pair, count))

        # Sắp xếp các cặp theo tần suất cao nhất
        related_pairs.sort(key=lambda x: x[1], reverse=True)

        # Lọc cặp có tần suất cao nhất và gợi ý sản phẩm
        if related_pairs:
            pair, _ = related_pairs[0]  # Lấy cặp có tần suất cao nhất
            # Gợi ý sản phẩm từ cặp (không phải sản phẩm đã xem)
            if pair[0] != product:
                recommendations[pair[0]] = product
            else:
                recommendations[pair[1]] = product

    return list(recommendations.keys())

# Hàm lưu gợi ý vào database mới
```

```
def save_recommendations(session_id, recommendations):
    # Lưu vào collection Recommendations trong ProductRecommendationsDB
    if recommendations:
                data = {"session ID": session_id, "recommendations":
recommendations}
        output_collection.insert_one(data)
                print(f"Recommendations for session {session_id} saved
successfully.")
    else:
        print(f"No recommendations found for session {session_id}.")

# Lấy tất cả sản phẩm người dùng đã xem và cặp sản phẩm từ Frequency
viewed_products_by_session = get_all_viewed_products()
all_product_pairs = get_all_product_pairs()

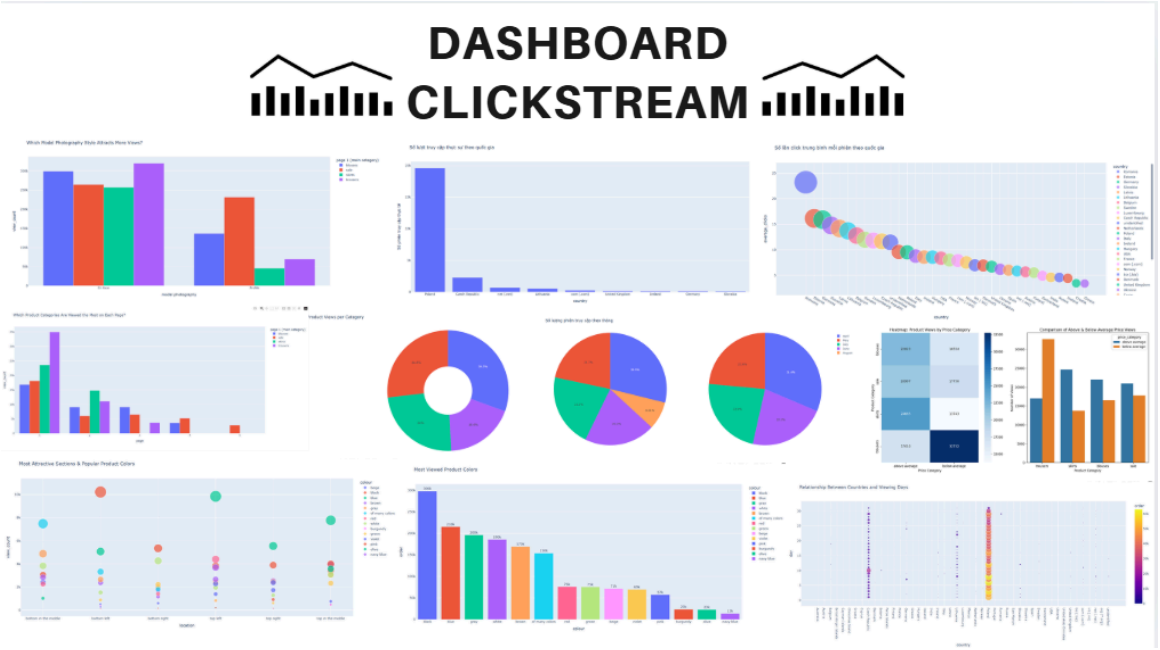
# Lấy gợi ý cho tất cả session ID
for session_id, viewed_products in viewed_products_by_session.items():
             recommendations = get_recommendations(viewed_products,
all_product_pairs)
    save_recommendations(session_id, recommendations)
```

3.2. Demo ứng dụng

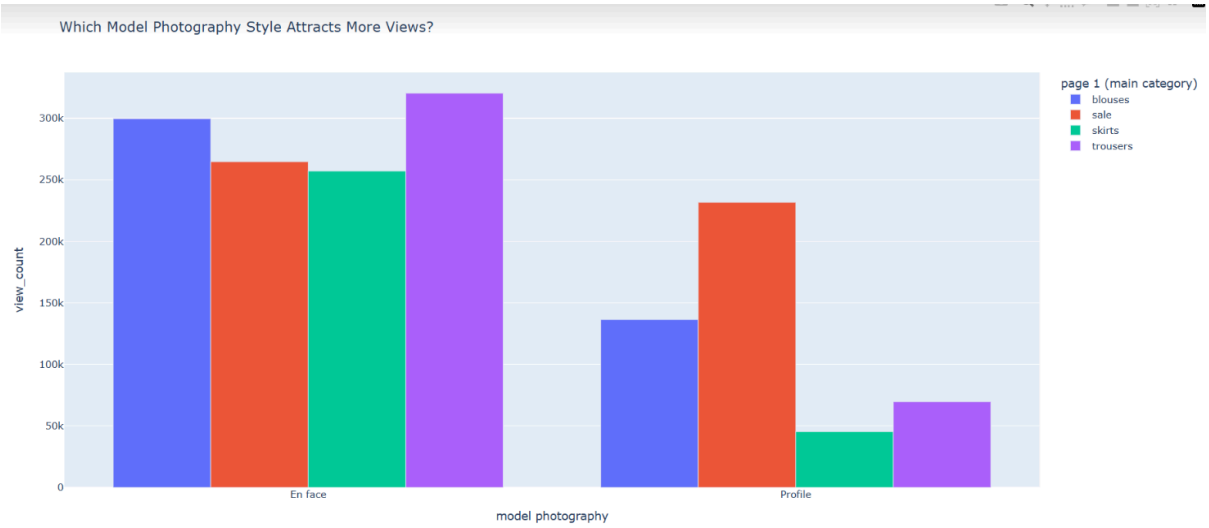
Các File code liên quan cho phần mô tả thuật toán cùng với phần Demo chi tiết về bài toán được nhóm gộp chung và đẩy lên Drive để có thể tiện theo dõi
Link drive: CÁC FILE CẦN THIẾT

3.3. Phân tích, giải thích kết quả thu được

3.3.1. Phân tích, giải thích kết quả phần trực quan hóa



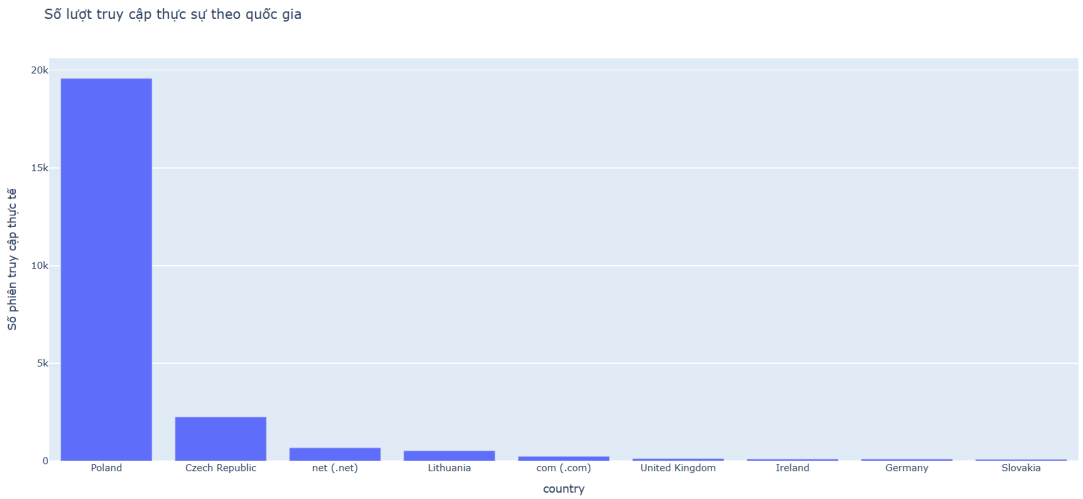
Hình 15: Bảng kết quả trực quan hóa tổng quan



Hình 16: Biểu đồ minh họa sự phân bố lượt xem dựa trên kiểu chụp

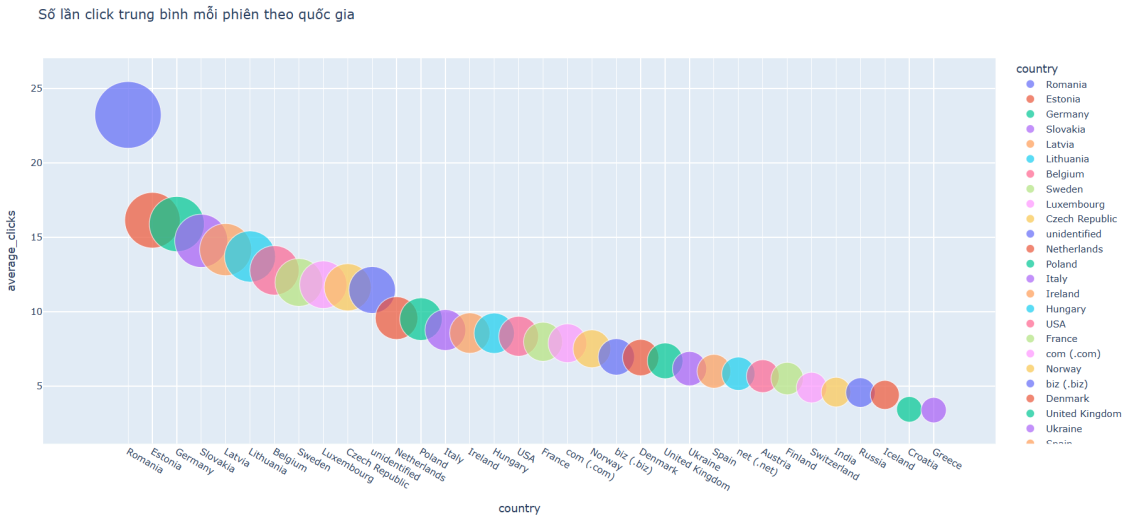
Biểu đồ minh họa sự phân bố lượt xem của các kiểu chụp ảnh mẫu trong bốn danh mục thời trang chính: "blouses" (áo blouse), "sale" (hàng giảm giá), "skirts" (chân váy) và "trousers" (quần dài). Trục hoành thể hiện hai kiểu chụp ảnh: "En face" (chụp trực diện) và "Profile" (chụp nghiêng), trong khi trục tung biểu diễn số lượt xem (view_count). Từ biểu đồ, ta có thể thấy rằng ảnh chụp trực diện (En face) thu hút sự chú ý hơn hẳn so với ảnh chụp nghiêng (Profile) trong tất cả các danh mục thời trang. Đặc biệt, danh mục "trousers" (màu tím) dẫn đầu về lượt xem trong kiểu chụp trực

diện, chứng tỏ sự quan tâm mạnh mẽ của khán giả đối với hình ảnh này. Điều này có thể do cách chụp trực diện làm nổi bật chi tiết và phom dáng của quần dài, tạo ấn tượng sâu sắc hơn cho người xem. Ngược lại, trong kiểu chụp nghiêng, danh mục "sale" (màu đỏ) lại chiếm ưu thế với số lượt xem cao nhất. Điều này cho thấy sức hấp dẫn của hàng giảm giá khi được thể hiện qua góc chụp nghiêng, có lẽ nhờ vào cảm giác mới lạ và độc đáo mà góc chụp này mang lại. Trái ngược hoàn toàn, danh mục "skirts" (màu xanh lá) lại nhận được ít sự quan tâm nhất trong ảnh chụp nghiêng. Nguyên nhân có thể là do kiểu chụp nghiêng không làm nổi bật được kiểu dáng và chi tiết của chân váy, dẫn đến việc giảm sức hút đối với người xem.



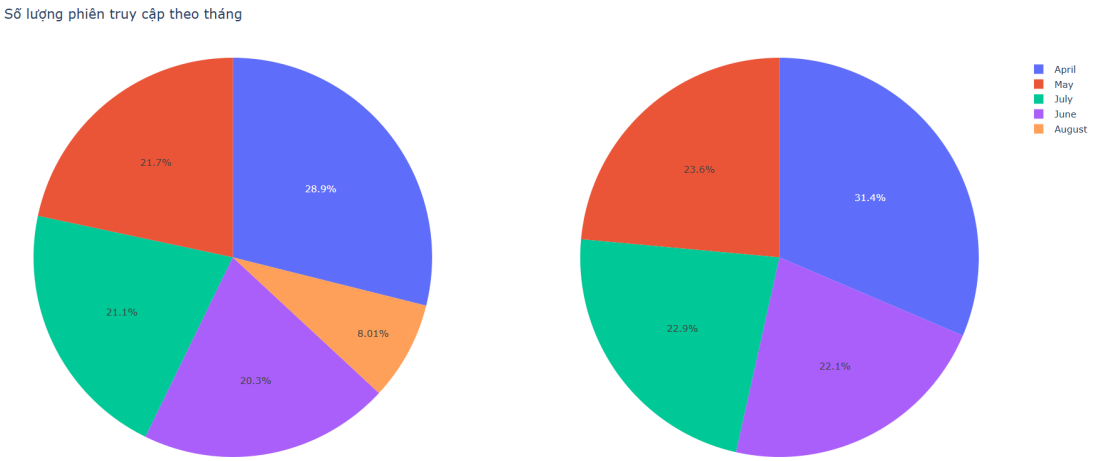
Hình 17: Biểu đồ minh họa số lượt truy cập thực sự theo quốc gia

Biểu đồ trên minh họa số phiên truy cập thực tế bao gồm 2 phần chính là "Quốc gia" và "Tên Miền". Trong đó, Ba Lan (Poland) có số lượt truy cập cao nhất, vượt xa các quốc gia khác, với khoảng 20.000 lượt. Thì Cộng hòa Séc (Czech Republic) đã đứng thứ hai nhưng số lượt truy cập thấp hơn đáng kể so với Ba Lan. Về các quốc gia còn lại như Lithuania, Anh, Ireland, Đức, Slovakia thể hiện một lượng truy cập khá thấp. Bên cạnh đó, tên các miền chung như .net, .com cũng có cho thấy 1 số liệu khá nhỏ về lượng người dùng truy cập. Nhiều nhất là .net theo sau là .com và một vài miền khác.



Hình 18: Biểu đồ minh họa số lần click trung bình mỗi phiên theo quốc gia

Biểu đồ hiển thị số lần click trung bình mỗi phiên theo quốc gia cho thấy sự khác biệt rõ rệt về mức độ tương tác giữa các khu vực. Trục hoành liệt kê các quốc gia như Romania, Estonia, Germany, Slovakia, Latvia, Lithuania, cùng với một số miền chung như .com, .net, .biz. Trục tung biểu diễn số lần click trung bình mỗi phiên. Kích thước của các bong bóng biểu thị mức độ quan trọng hoặc số lượng dữ liệu tương ứng với từng quốc gia. Romania nổi bật với số lần click trung bình cao nhất, vượt mức 25 click mỗi phiên và có kích thước bong bóng lớn nhất. Điều này cho thấy sự tương tác mạnh mẽ từ người dùng tại quốc gia này. Các quốc gia như Estonia, Germany, Slovakia, Latvia, và Lithuania có mức click trung bình nằm trong khoảng 15-20 click mỗi phiên, thể hiện mức độ hoạt động ổn định. Ngược lại, các quốc gia như Anh, Tây Ban Nha, Ukraine, và Áo lại có số lần click trung bình thấp hơn, dưới 10 click mỗi phiên, cho thấy mức độ tương tác ít hơn. Bên cạnh đó, một số miền chung như .com, .net, và .biz cũng xuất hiện nhưng có mức click trung bình tương đối thấp, cho thấy sự phân tán dữ liệu trên các miền này.



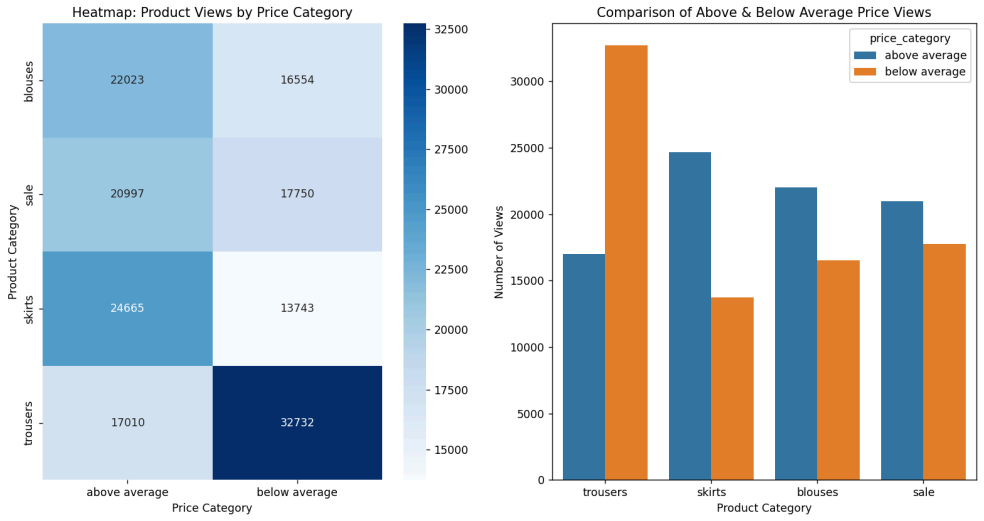
Hình 19: Biểu đồ minh họa số lượng phiên truy cập theo tháng

Hình ảnh trên hiển thị hai biểu đồ tròn phân tích số lượng phiên truy cập theo tháng, với từng màu sắc đại diện cho một tháng trong năm, bao gồm Tháng 4, Tháng 5, Tháng 6, Tháng 7 và Tháng 8. Tỷ lệ phần trăm trên mỗi phần biểu đồ thể hiện mức đóng góp của từng tháng vào tổng số phiên truy cập.

Biểu Đồ Bên Trái cho thấy Tháng 4 chiếm tỷ lệ cao nhất với 28.9%, cho thấy đây là thời điểm có sự gia tăng đáng kể trong số lượng phiên truy cập. Ngược lại, Tháng 8 có số lượng truy cập thấp nhất với chỉ 8.01%, cho thấy sự sụt giảm mạnh mẽ trong tháng này. Các tháng khác như Tháng 5 (21.7%), Tháng 7 (21.1%), và Tháng 6 (20.3%) có mức truy cập tương đối đồng đều, cho thấy sự ổn định trong các tháng này.

Biểu Đồ Bên Phải tiếp tục khẳng định rằng Tháng 4 vẫn là tháng dẫn đầu về lượng truy cập với tỷ lệ 31.4%. Điều đáng chú ý là Tháng 8 không xuất hiện trong biểu đồ này, có thể do dữ liệu đã được lọc hoặc loại bỏ, tạo ra sự khác biệt đáng kể so với biểu đồ bên trái. Các tháng còn lại như Tháng 5 (23.6%), Tháng 7 (22.9%), và Tháng 6 (22.1%) duy trì mức truy cập khá đồng đều, thể hiện sự phân bổ hợp lý trong các hoạt động truy cập.

Qua phân tích trên, có thể thấy được Tháng 4 là tháng có lượng truy cập cao nhất trong cả hai biểu đồ, trong khi Tháng 8 có mức độ tương tác thấp hơn hoặc không xuất hiện trong dữ liệu biểu đồ bên phải. Điều này cho thấy sự biến động trong xu hướng truy cập theo tháng và cần có sự tập trung vào Tháng 4 để tối ưu hóa các chiến dịch hoặc hoạt động liên quan.



Hình 20: Biểu đồ minh họa số lượng truy cập theo các phân loại giá cả khác nhau

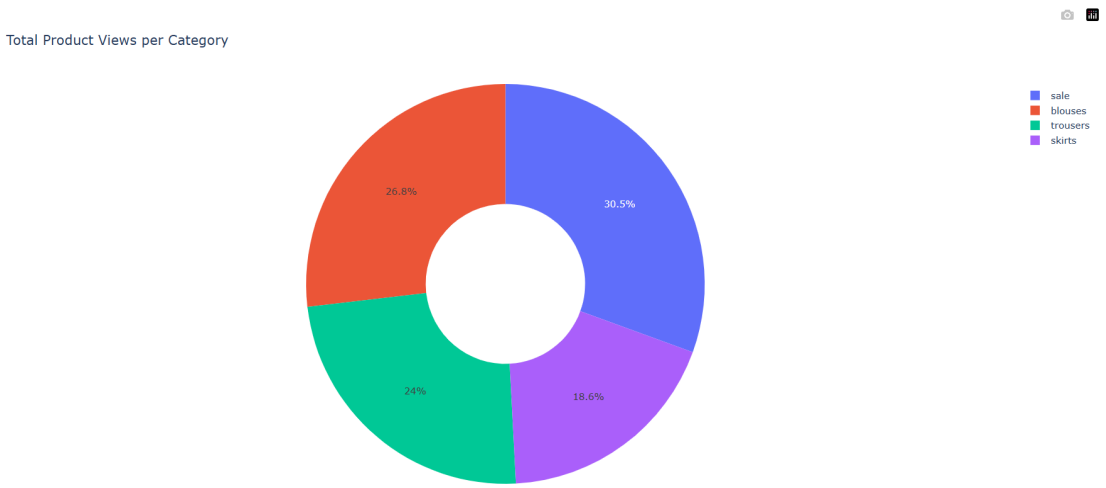
Hình ảnh đi kèm bao gồm hai biểu đồ trực quan hóa số lượt xem sản phẩm theo từng danh mục giá.

Biểu đồ Heatmap (bên trái) thể hiện số lượt xem của mỗi danh mục sản phẩm, phân chia theo hai nhóm giá: "Giá trên trung bình" và "Giá dưới trung bình". Sản phẩm

"trousers" (quần dài) đạt số lượt xem cao nhất trong nhóm giá thấp với 32,732 lượt, trong khi ở nhóm giá cao, chỉ đạt 17,010 lượt. Sản phẩm "skirts" (chân váy) cho thấy sự khác biệt đáng kể giữa hai nhóm giá, với 24,665 lượt xem ở nhóm giá cao và chỉ 13,743 lượt xem ở nhóm giá thấp. "Blouses" (áo kiểu) và "sale" (giảm giá) duy trì số lượt xem khá ổn định giữa cả hai nhóm giá.

Biểu đồ Cột (bên phải) so sánh số lượt xem giữa hai nhóm giá cho từng danh mục sản phẩm: "Trousers" thể hiện số lượt xem vượt trội ở nhóm giá thấp, điều này chỉ ra rằng người tiêu dùng có xu hướng quan tâm nhiều hơn đến các sản phẩm giá rẻ trong danh mục này. "Skirts" có số lượt xem cao hơn trong nhóm giá cao, cho thấy khách hàng ưa chuộng các sản phẩm chất lượng cao trong danh mục này. "Blouses" và "sale" có số lượt xem nhóm giá cao nhiều hơn nhóm giá thấp, tuy nhiên, không có sự chênh lệch đáng kể.

Tổng quan chung, người tiêu dùng dường như ưa thích quần dài giá rẻ, trong khi váy có xu hướng hấp dẫn hơn ở mức giá cao hơn. Các sản phẩm áo và sản phẩm giảm giá có mức độ quan tâm tương đối đồng đều giữa hai mức giá. Những dữ liệu này có thể hỗ trợ các nhà bán lẻ trong việc điều chỉnh chiến lược giá và quảng cáo cho từng danh mục sản phẩm một cách hiệu quả



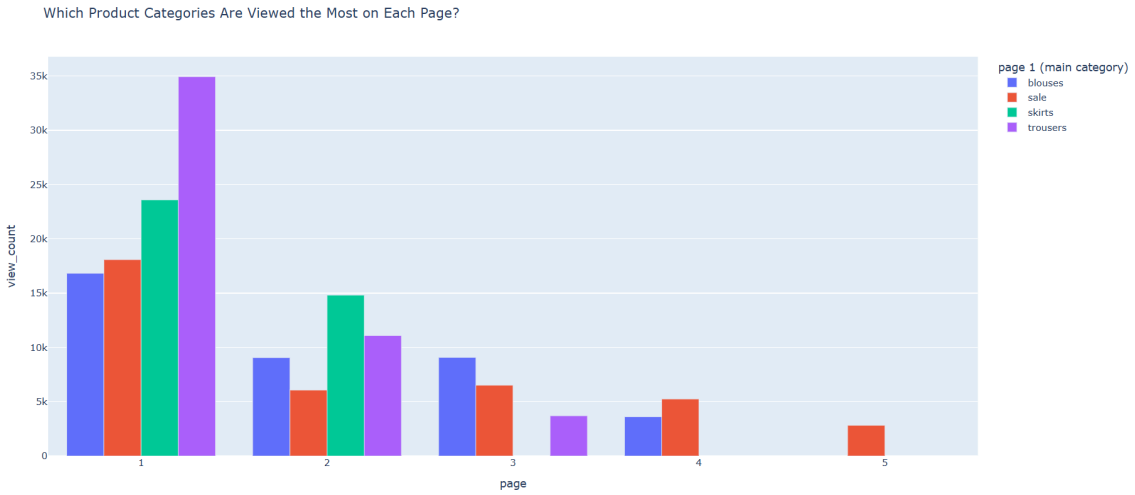
Hình 21: Biểu đồ minh họa tổng lượt xem theo từng danh mục sản phẩm

Hình ảnh biểu đồ tròn dạng donut trên minh họa tổng số lượt xem sản phẩm theo từng danh mục, cung cấp cái nhìn sâu sắc về xu hướng tiêu dùng hiện tại.

+ Danh mục "sale" (màu xanh dương) chiếm ưu thế với 30.5% tổng số lượt xem, phản ánh sự hấp dẫn đặc biệt của các sản phẩm giảm giá đối với khách hàng. Điều này có thể xuất phát từ các chính sách khuyến mãi hiệu quả hoặc tâm lý tiêu dùng ưa chuộng hàng giảm giá.
+ Danh mục "blouses" (màu đỏ) chiếm 26.8%, giữ vị trí thứ hai, cho thấy sự quan tâm đáng kể từ phía người tiêu dùng đối với loại sản phẩm này.

+ Danh mục "trousers" (màu xanh lá cây), với 24% tổng số lượt xem, không chênh lệch nhiều so với "blouses", gợi ý rằng đây cũng là một danh mục sản phẩm phổ biến và thu hút sự chú ý.
+ Danh mục "skirts" (màu tím) có lượt xem ít nhất, chiếm 18.6%, có thể do số lượng lựa chọn hạn chế hơn hoặc không nằm trong danh mục ưu tiên của người mua.

Có thể kết luận rằng dữ liệu này chỉ ra rằng sản phẩm giảm giá là yếu tố thu hút chính, có lẽ nhờ vào các yếu tố khuyến mãi hoặc tâm lý tiêu dùng thích mua hàng ưu đãi. Blouses và trousers cho thấy mức độ quan tâm tương đương, cho thấy đây là hai danh mục sản phẩm phổ biến. Skirts, với lượt xem thấp nhất, có thể không phải là lựa chọn ưu tiên của khách hàng hoặc ít có sự lựa chọn. Thông tin này giúp các nhà bán lẻ hiểu rõ hơn về sở thích mua sắm của khách hàng, từ đó tối ưu hóa danh mục sản phẩm và chiến lược tiếp thị của mình.



Hình 22: Biểu đồ minh họa số lượt xem của các danh mục sản phẩm trên từng trang

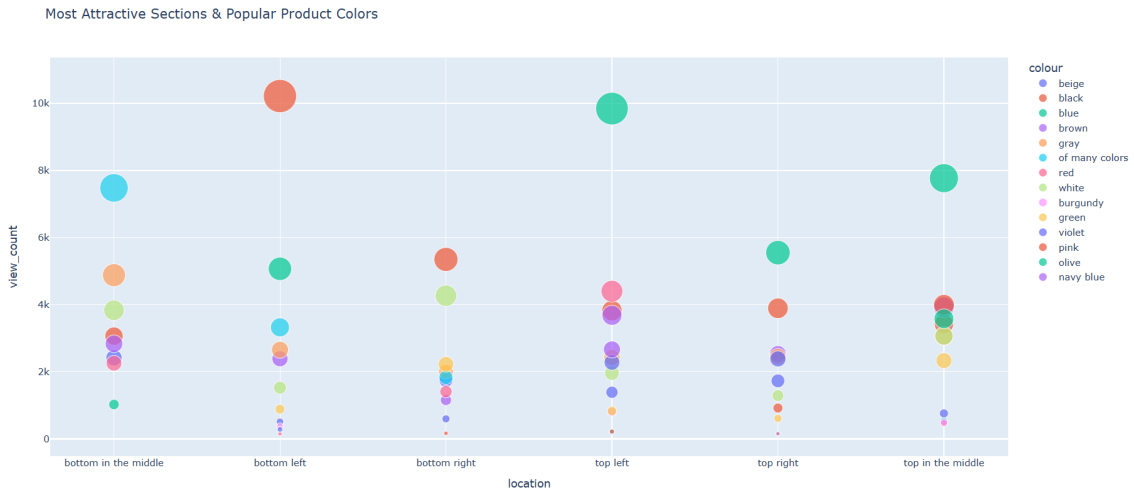
Biểu đồ cột trên thể hiện số lượt xem của các danh mục sản phẩm trên từng trang, trong đó có bốn danh mục sản phẩm chính và được phân biệt bằng màu sắc: Áo blouse (blouses) - cột màu xanh dương, Sản phẩm giảm giá (sale) - cột màu cam, Chân váy (skirts) - cột màu xanh và Quần (trousers) - cột màu tím.

Dựa vào biểu đồ, có thể thấy trang 1 có số lượt xem cao nhất so với 3 trang còn lại. Trong đó, Quần có lượt xem cao nhất với 35k lượt xem, tiếp theo đó là Chân váy với 25k lượt xem. Mặt khác, Sản phẩm giảm giá và Áo blouse có lượt xem thấp hơn nhưng vẫn trên 15k. Đến với trang thứ 2, có thể thấy sự thay đổi về lượt xem giữa các sản phẩm thay đổi rõ rệt. Ở trang này, Chân váy lại dẫn đầu với xấp xỉ 15k lượt xem, theo sau là Quần với xấp xỉ, trong khi Blouses và Sale giảm xuống còn xấp xỉ 9k và 6k lượt xem.

Đối với trang 3 và 4, có thể thấy được lượng tương tác và số lượt xem giảm đáng kể so với hai trang đầu. Trước tiên, ở trang 3, Áo blouse và Sản phẩm giảm giá có lượt xem khoảng 8k và 6k lượt, trong khi Quần và Chân váy có lượng xem rất thấp. Bên cạnh đó, ở trang trang 4, Sản phẩm giảm giá có lượt xem cao nhất, khoảng 6.000 lượt, vượt trội so với các danh mục còn lại, trong khi Áo blouse, Chân váy và Quần có lượt xem khá thấp. Tuy nhiên, khi quan sát lượt xem ở trang 5, chỉ có danh mục Sản phẩm giảm giá xuất hiện với một lượng nhỏ lượt xem, khoảng 3k lượt, còn các danh mục khác không hiển thị dữ liệu.

Qua biểu đồ này, có thể thấy trang đầu tiên có xu hướng thu hút nhiều lượt xem nhất, đặc biệt là Quần và Chân váy. Tuy nhiên, lượt xem có xu hướng giảm dần qua từng trang, càng về sau càng ít. Mặt khác, Sản phẩm giảm giá vẫn duy trì lượng xem ổn định trên các trang, ngay cả ở trang 4 và 5. Điều này cho thấy người dùng vẫn quan tâm đến sản phẩm khuyến mãi ngay cả khi phải duyệt nhiều trang hơn. Trong số các danh mục sản phẩm, Quần chiếm số lượt xem cao nhất, cho thấy sự phổ biến của danh mục này. Bên cạnh đó, Áo blouse và Chân váy thu hút sự quan tâm cao trên trang đầu nhưng lại giảm mạnh ở các trang sau.

Từ những phân tích trên, có thể thấy được các sản phẩm phổ biến thường xuất hiện trên trang đầu, nhằm thu hút được nhiều lượt xem hơn. Do đó, nếu có mục tiêu là tối ưu hóa lượt xem, việc hiển thị các danh mục phổ biến như Quần và Chân váy trên trang đầu sẽ mang lại hiệu quả cao hơn. Một điều đáng chú ý khác có thể thấy được là Sản phẩm giảm giá vẫn giữ được sức hút ngay cả khi nằm ở các trang sau.



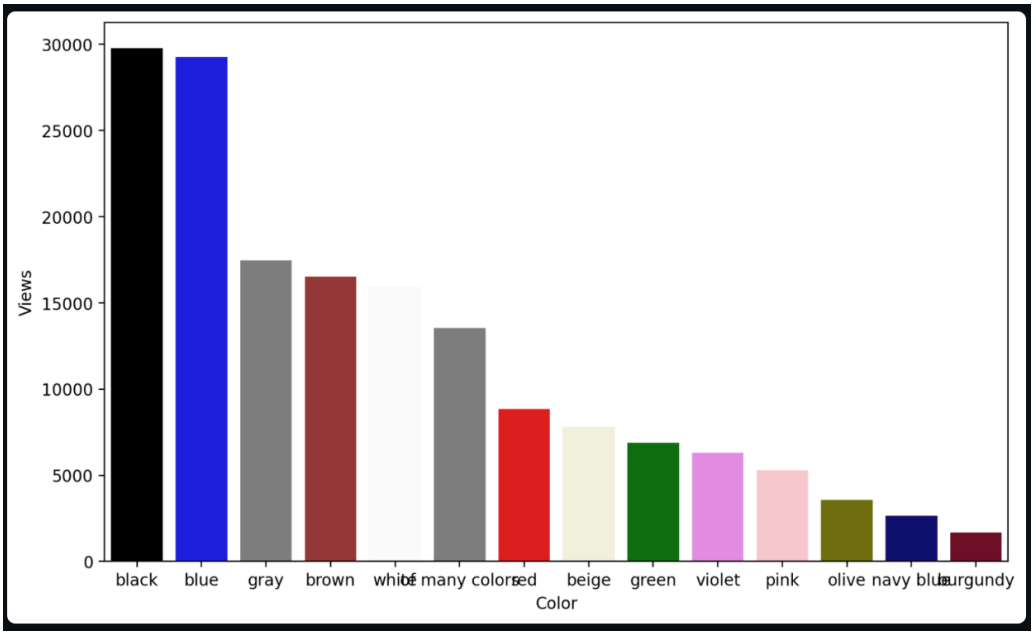
Hình 23: Biểu đồ minh họa gian hàng thu hút và màu sắc sản phẩm phổ biến nhất

Hình ảnh hiển thị là biểu đồ bong bóng minh họa các vị trí nổi bật nhất trên trang web cùng với sự phổ biến của các màu sắc sản phẩm.

+ **Trục Hoành (Location):** Biểu diễn các vị trí khác nhau trên trang web, bao gồm "bottom in the middle", "bottom left", "bottom right", "top left", "top right", và "top in the middle".
+ **Trục Tung (View Count):** Thể hiện số lượt xem tương ứng với từng vị trí.

+ **Màu Sắc Bong Bóng:** Đánh dấu các màu sắc sản phẩm khác nhau như beige, black, blue, green, red, white, pink, navy blue,...
+ **Kích Thước Bong Bóng:** Phản ánh mức độ phổ biến của sản phẩm tại vị trí cụ thể dựa trên màu sắc.

Cụ thể thì vị trí "top left" và "bottom in the middle" ghi nhận lượng xem cao nhất, với các bong bóng có kích thước lớn, chỉ ra rằng đây là những khu vực thu hút sự chú ý đáng kể trên trang web. Bên cạnh đó, màu sắc sản phẩm có lượng xem cao bao gồm xanh lá (green), đỏ (red), và xanh dương (blue), đặc biệt là tại các vị trí nổi bật như "top left" và "top right". Còn các vị trí "bottom left" và "bottom right" thể hiện số lượt xem thấp hơn, tuy nhiên vẫn có sự hiện diện của một số màu sắc phổ biến. Nhìn chung, những phân tích này cung cấp cái nhìn sâu sắc về xu hướng người dùng và sở thích màu sắc, từ đó hỗ trợ chiến lược tiếp thị hiệu quả hơn trên trang web.



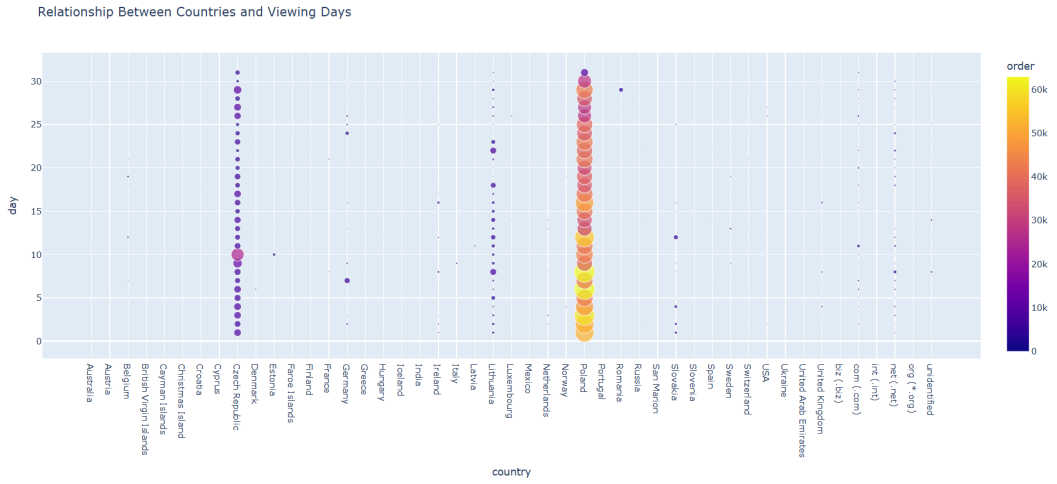
Hình 24: Biểu đồ minh họa số lượng đơn đặt hàng theo màu sắc sản phẩm.

Biểu đồ cột trên thể hiện số lượng đơn đặt hàng theo màu sắc sản phẩm, trong đó gồm 2 phần chính là “order” (lượng đặt hàng) và “colour” (màu sắc sản phẩm). Dựa vào số lượng đặt hàng trên đồ thị trên, ta có thể chia các màu thành các nhóm như sau: nhóm màu sắc sản phẩm được ưa chuộng nhất, ưa chuộng trung bình và ít được ưa chuộng.

Nhóm màu sắc sản phẩm được ưa chuộng nhất là các màu sắc như đen, xanh dương, xám, trắng, nâu và màu sắc hỗn hợp với số đặt hàng trên 150k đơn cho mỗi màu. Trong đó, màu đen là màu phổ biến nhất với 300k đơn hàng, cao hơn đáng kể so với các màu khác. Trong khi đó, các màu sắc mang khuynh hướng rực rỡ hơn như đỏ, xanh lá, be, tía và hồng có số lượng đặt hàng trung bình và ít được ưa chuộng hơn so với các màu trung tính. Màu đỏ, màu sắc có số lượng đặt hàng cao nhất trong số các màu được ưa chuộng trung bình lại có số lượng đặt hàng thấp hơn gấp đôi, thậm chí

gấp ba so với các màu được ưa chuộng nhất. Tuy nhiên, đặc biệt nhất là các màu hiếm như burgundy, olive và navy blue có số đơn hàng thấp nhất với dưới 25k đơn hàng, vì thế đây là những màu ít được ưa chuộng.

Qua đồ thị trên, có thể thấy thấy người tiêu dùng có xu hướng ưu tiên lựa chọn các màu sắc cơ bản, dễ phối hợp hơn các màu sặc sỡ hoặc hiếm. Tuy nhiên, điều bất ngờ nhất là sản phẩm có màu sắc hỗn hợp có thứ hạng khá cao, với 150k đơn hàng. Điều này đã cho thấy một số khách hàng vẫn yêu thích những sản phẩm có sự đa dạng về màu sắc.



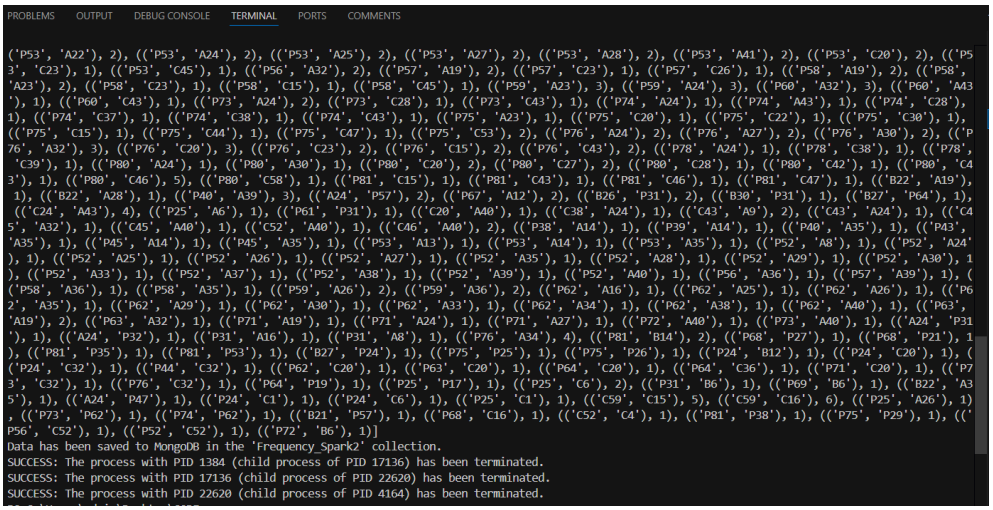
Hình 25: Biểu đồ minh họa mối quan hệ giữa các nước và số ngày theo dõi

Biểu đồ trên thể hiện mối quan hệ giữa quốc gia và số ngày xem (day) với số lượng đơn đặt hàng (order), được biểu thị bằng kích thước và màu sắc của các điểm dữ liệu. Trục hoành (X) biểu thị tên quốc gia, bao gồm nhiều quốc gia trên thế giới, trong khi trục tung (Y) đại diện cho số ngày xem. Màu sắc và kích thước của các điểm thể hiện số lượng đơn đặt hàng, với màu vàng và kích thước lớn hơn biểu thị số lượng đơn hàng cao. Quan sát xu hướng, phần lớn các quốc gia có số lượng đặt hàng nhỏ, thể hiện qua các điểm nhỏ màu tím, trong khi một số quốc gia như Ba Lan và Bồ Đào Nha có lượng đặt hàng cao hơn, với các điểm lớn màu vàng và cam. Số ngày xem phân bố khá đồng đều trên các quốc gia nhưng có sự khác biệt rõ rệt về số lượng đơn đặt hàng. Đáng chú ý, Ba Lan là quốc gia có số lượng đơn đặt hàng cao nhất, thể hiện qua cụm điểm lớn màu vàng, tiếp theo là Bồ Đào Nha, trong khi các quốc gia khác như Đức, Mỹ và Vương quốc Anh có sự phân tán nhưng không nổi bật bằng hai quốc gia trên. Ngoài ra, có một nhóm dữ liệu thuộc danh mục “Unidentified” hoặc các miền tổ chức như org, net, com, cho thấy một số người dùng không xác định được quốc gia.

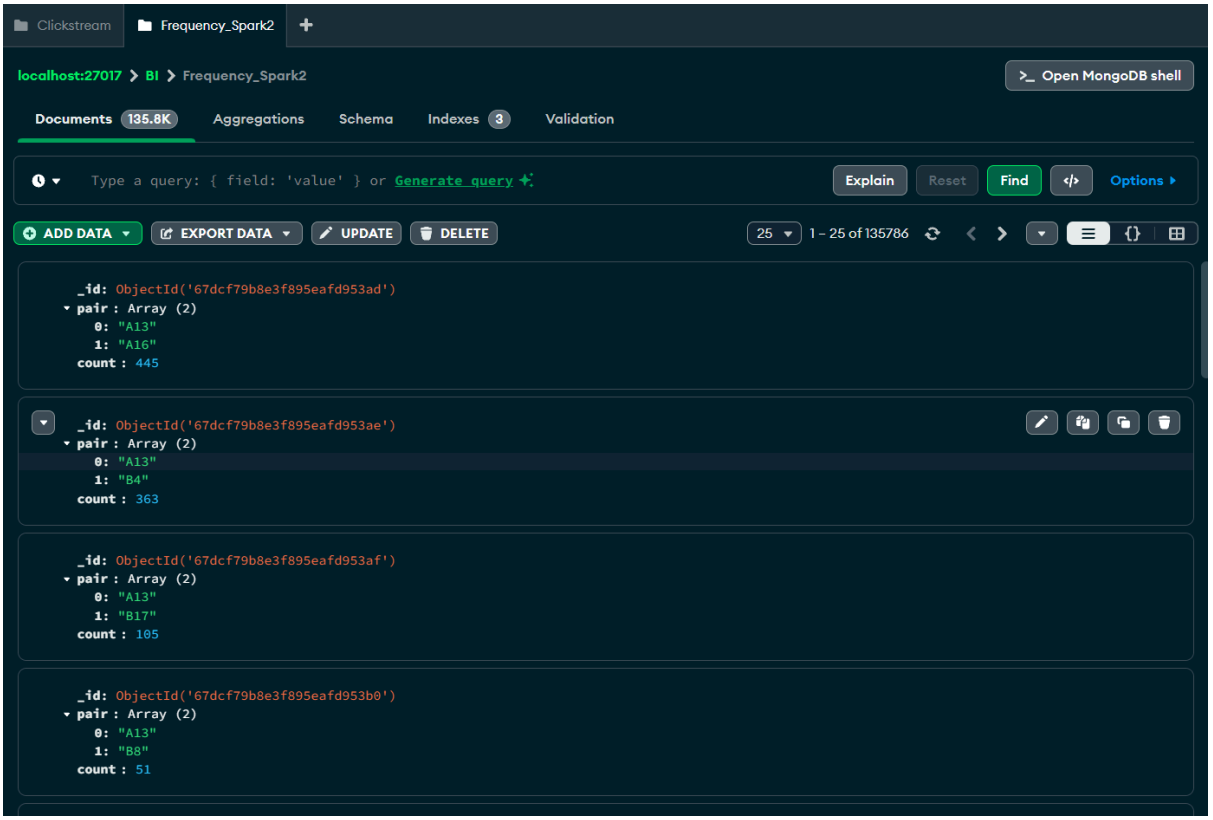
Kết luận, Ba Lan và Bồ Đào Nha là hai quốc gia có lượng đơn đặt hàng lớn nhất, trong khi các quốc gia còn lại có mức đặt hàng thấp hơn và phân bố rải rác. Màu sắc trên biểu đồ giúp dễ dàng xác định các khu vực có lượng đơn hàng cao, tuy nhiên sự xuất hiện của dữ liệu không xác định có thể ảnh hưởng đến độ chính xác của phân tích.

3.3.2. Kết quả phân tích đề xuất bằng PySpark

Bước 1: Chạy MapReduce bằng thư viện PySpark và lưu kết quả vào collection "Frequency_Spark2". Tại đây thuật toán sẽ ghép cặp lần lượt từng sản phẩm mà mỗi người dùng đã xem trong từng "session ID", sau đó sẽ tổng hợp lại tần suất của mỗi cặp sản phẩm đã được ghép. Kết quả sẽ được lưu vào MongoDB



Hình 26. Kết quả thuật toán khi chạy code

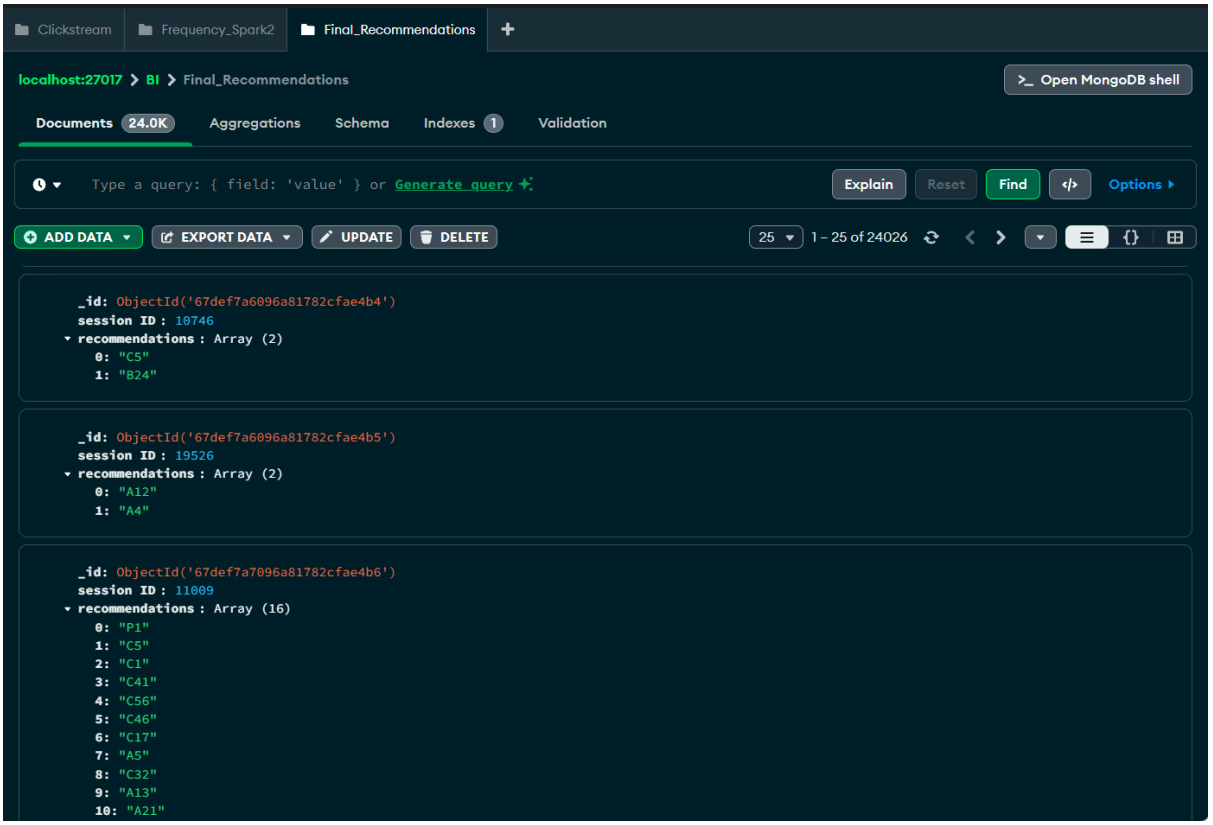


Hình 27. Kết quả thuật toán MapReduce được lưu vào MongoDB

	A	B	C	D
1	_id	pair[0]	pair[1]	count
2	67dee5bae	A2	A5	1226
3	67dee5bae	A2	A3	1052
4	67dee5bae	A2	A10	1019
5	67dee5bae	A1	A2	956
6	67dee5bae	A10	A12	910
7	67dee5bae	A3	A5	893
8	67dee5bae	A9	A10	886
9	67dee5bae	A10	A10	878
10	67dee5bae	A2	A7	871
11	67dee5bae	A5	A10	870
12	67dee5bae	A2	A11	852
13	67dee5bae	A2	A9	833
14	67dee5bae	A2	A8	829
15	67dee5bae	A2	A12	801
16	67dee5bae	A2	A6	775
17	67dee5bae	A5	A12	760
18	67dee5bae	A9	A12	753
19	67dee5bae	A5	A9	751
20	67dee5bae	A5	A8	738
21	67dee5bae	A5	A6	737
22	67dee5bae	A6	A9	732
23	67dee5bae	P1	P1	717
24	67dee5bae	A5	A7	715
25	67dee5bae	A5	A5	711
26	67dee5bae	A6	A10	709
27	67dee5bae	A3	A10	704

Hình 28: Kết quả thuật toán MapReduce được xuất ra CSV

Bước 2: Thực hiện đề xuất. Tại đây thuật toán sẽ truy vấn lấy danh sách sản phẩm mà người dùng (mỗi "session ID) đã xem, sau đó xuất dựa sản phẩm được ghép cặp với mỗi sản phẩm mà có tần suất cao nhất cho người dùng. Kết quả sẽ được lưu vào collection "Final_Recommendations"

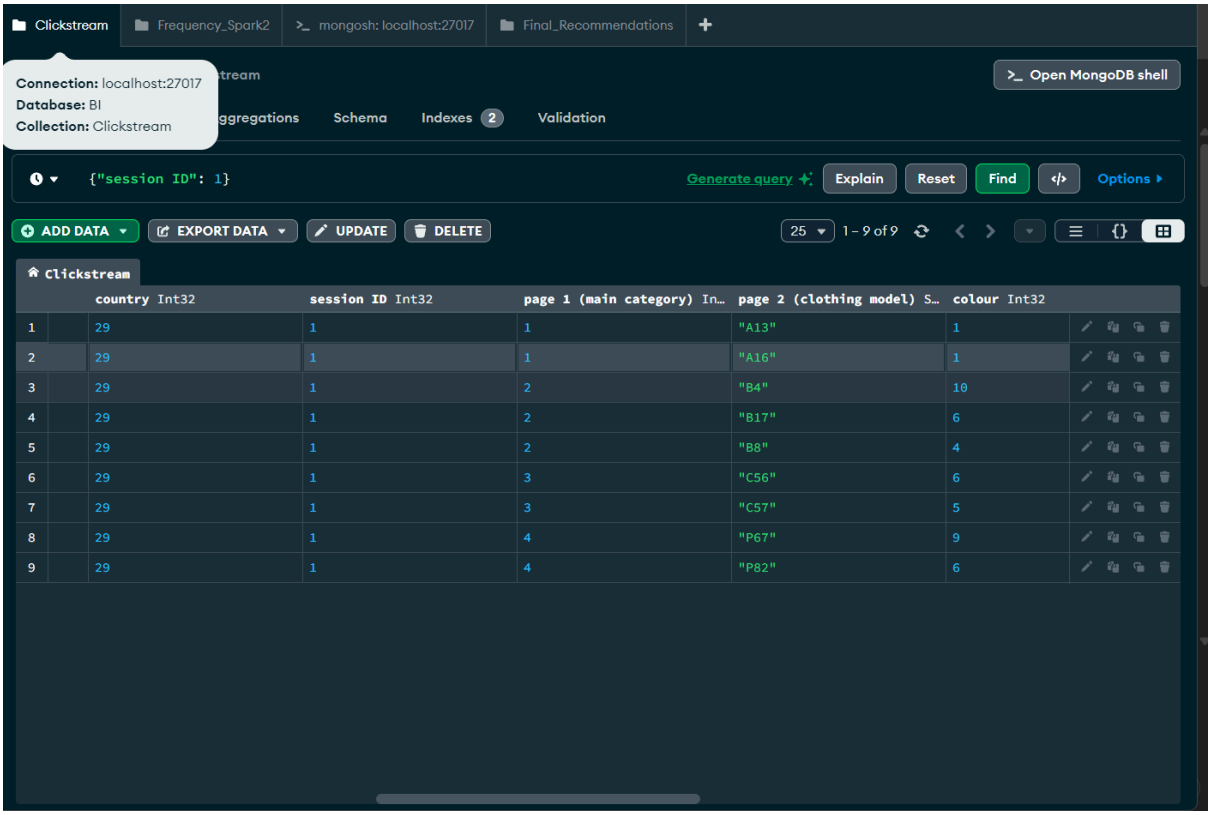


Hình 29. Kết quả đề xuất dựa trên MapReduce được lưu vào MongoDB

	A	B	C	D	E	F	G	H	I	J	K	L	M	N	O	P	Q
1	_id	session ID	recommer	recommer	recommer	recommer	recommer	recommer	recommer	recommer	recommer	recommer	recommer	recommer	recommer	recommer	recommer
2	67def8df0	1	A21	B4	B16	C56	P1										
3	67defe9c0	2	B10	B24	A2	P1	P33	P16									
4	67def9a10	3	B16	C5	C1	P1											
5	67def96e0	4	A17	C5	C40												
6	67defa5b0	5	C17														
7	67def8430	6	C5	C17	B16	B24											
8	67def9f80	7	A2	C7	P49	P1	B4										
9	67deff750	8	C50	B24	C7	C1	A5	C29	C49	C5	B10						
10	67defdda0	9	A2	B24													
11	67defa270	10	A2	B4	B17	B16											
12	67df02790	11	A2														
13	67defb670	12	B4	B24	B10	C5	P16										
14	67df03ef0	13	A2	A5													
15	67defe390	14	A31														
16	67defdea0	15	A2	A5	B4	B10	B17	B16	B24	C17	C7	C56					
17	67defedb0	16	P82	P61													
18	67df020e0	17	C5	C56	A2	C35	B24	B10									
19	67df015a0	18	A12	A5	A2	A10	A11	A17	C5								
20	67def87c0	19	A2	C1	C7	C5	P1										
21	67def86e0	20	A2	A4	P1												
22	67deffa30	21	P1	B10	B13	C5	C7										
23	67def8950	22	A10	A2	A31	A11											
24	67defaf50	23	A13	B10	P1												
25	67def94d0	24	B4	B24	C5	P1											
26	67df00000	25	B16	C5													
27	67def7e20	26	A2														

Hình 30: Kết quả thuật toán đề xuất dựa trên MapReduce được xuất ra CSV

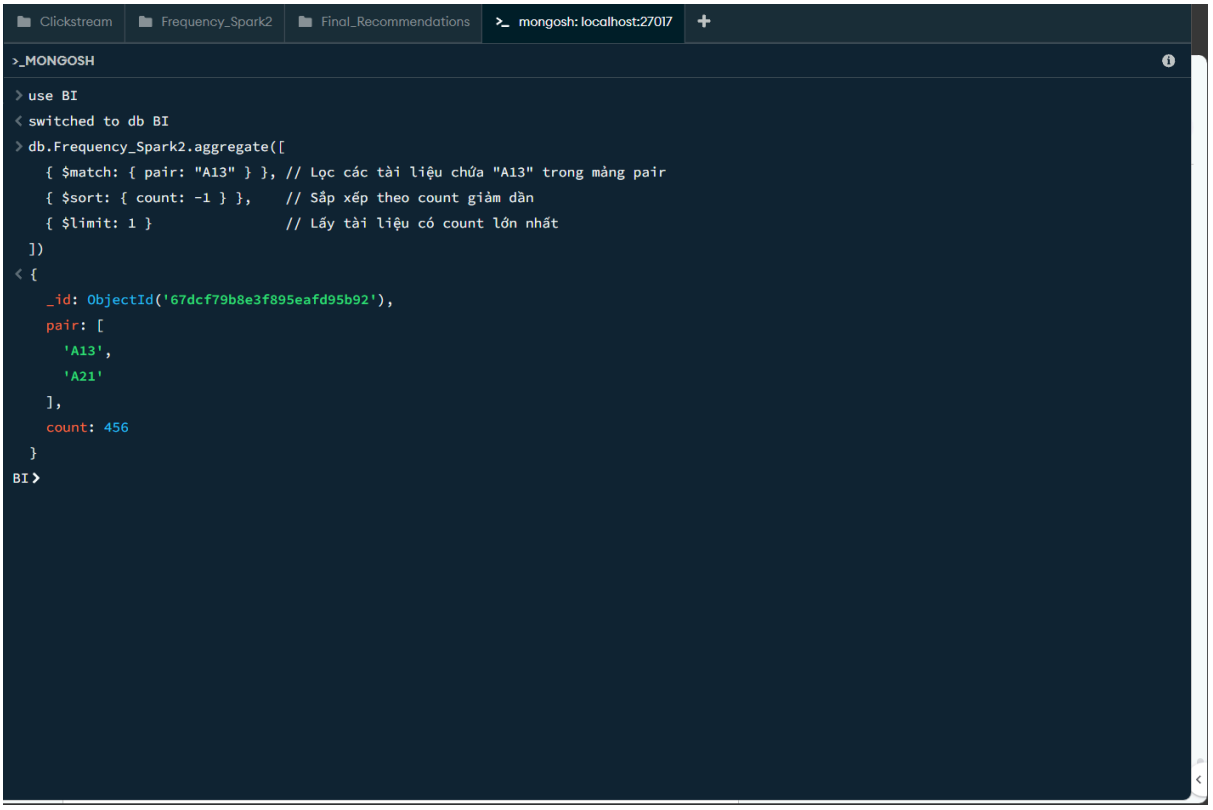
Bước 3: Kiểm nghiệm mô hình thuật toán. Tại đây sẽ kiểm tra thủ công bằng cách truy vấn trên dữ liệu trên một số người dùng nhất định. Đầu tiên sẽ truy vấn {"session ID": 1} trên collection “Clickstream” để xác định những sản phẩm mà người dùng đã xem.



Hình 31. Kết quả truy vấn {"session ID": 1}

Tiếp theo truy vấn sản phẩm được ghép cặp với từng sản phẩm mà có tần suất cao nhất bằng câu lệnh:

```
db.Frequency_Spark2.aggregate([
  { $match: { pair: "A13" } }, // Lọc các tài liệu chứa "A13" trong mảng pair (thay đổi theo từng sản phẩm)
  { $sort: { count: -1 } },    // Sắp xếp theo count giảm dần
  { $limit: 1 }                // Lấy tài liệu có count lớn nhất
])
```



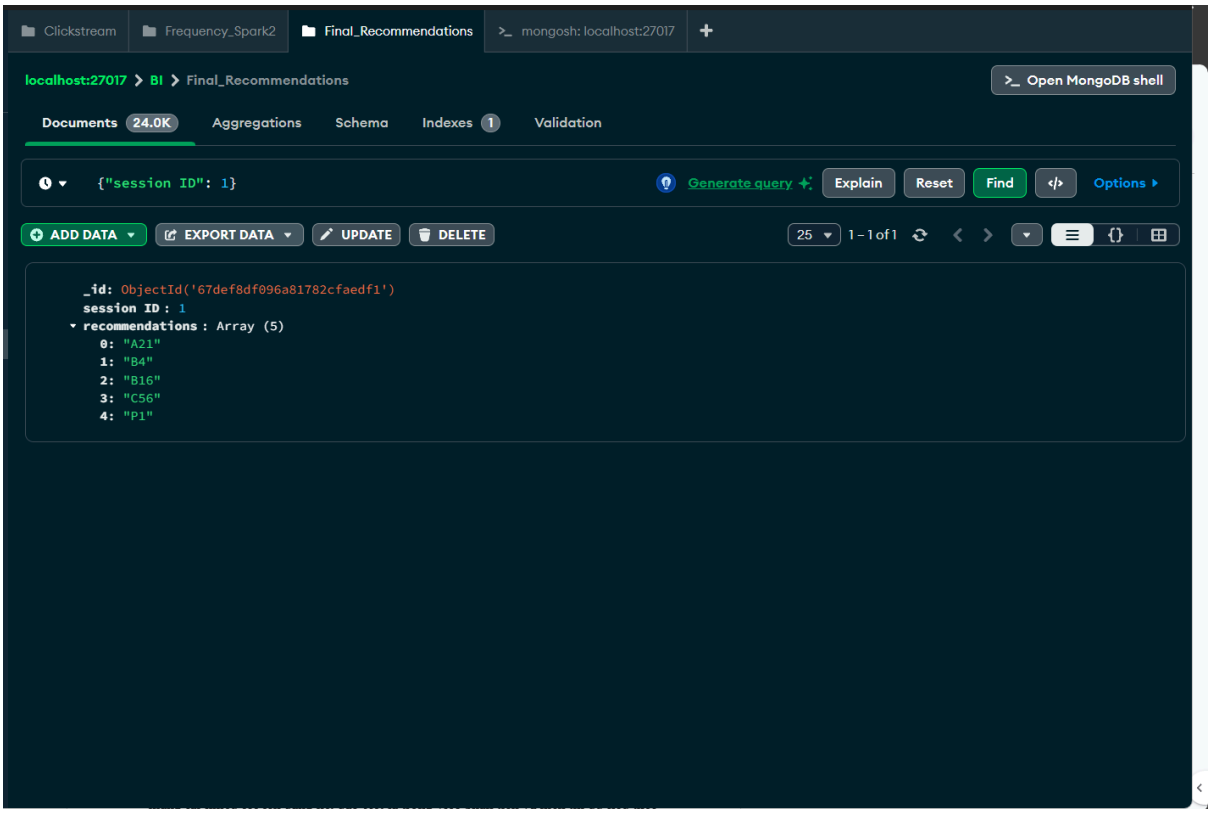
Hình 32. Kết quả truy vấn sản phẩm sẽ được đề xuất

Tương tự với các sản phẩm khác, ta được bảng như sau:

Bảng . Kết quả truy vấn các sản phẩm được ghép cặp có tần suất cao nhất

Sản phẩm đã xem	Sản phẩm được ghép cặp nhiều nhất
A13	A21
A16	A21
B4	B4
B17	B16
B8	B4
C56	C57
C57	C56
P67	P1
P82	P1

Cuối cùng, đối chiếu với đề xuất được hệ thống đề xuất. Có thể thấy hệ thống đã đề xuất đúng.



Hình 33. Kết quả đề xuất của thuật toán

CHƯƠNG 4: KẾT LUẬN BÀI TOÁN

4.1. Kết luận bài toán

Việc ứng dụng công cụ Big Data như PySpark và MongoDB giúp phân tích hành vi người dùng trên nền tảng thương mại điện tử diễn ra nhanh chóng và hiệu quả. Thông qua bộ dữ liệu Clickstream, nhóm chúng tôi đã xây dựng hệ thống có thể xác định các cặp sản phẩm thường được xem cùng nhau và từ đó đưa ra gợi ý sản phẩm phù hợp. Kết quả này không chỉ giúp cá nhân hóa trải nghiệm mua sắm mà còn tăng khả năng bán hàng và cải thiện doanh thu. Ngoài ra, việc lưu trữ và phân tích dữ liệu đồng thời trực quan hoá qua các dashboard giúp người dùng dễ dàng theo dõi xu hướng và tối ưu chiến lược kinh doanh.

4.2. Đánh giá ưu điểm, nhược điểm

4.2.1. Ưu điểm

Nhóm đã tích hợp các công nghệ Big Data vào hệ thống và đưa lên Streamlit mang lại nhiều lợi ích đáng kể, đặc biệt là trong việc phân tích và hiển thị dữ liệu theo hướng tiềm năng hỗ trợ hiển thị theo thời gian thực. Điều này cho phép hệ thống của chúng tôi có **khả năng xử lý khối lượng dữ liệu lớn từ nhiều nguồn khác nhau**. Ngoài ra, **đưa hệ thống lên Streamlit** mang lại lợi ích lớn về mặt hiển thị, giúp người dùng có thể truy cập dữ liệu thông qua một giao diện web đơn giản, trực quan mà không cần có kỹ năng lập trình chuyên sâu.Đồng thời, áp dụng các công cụ của Big Data giúp **giảm tải cho hệ thống back-end** cũng như tăng hiệu suất của hệ thống. Nhóm cũng sử dụng công cụ Map Reduce giúp tối ưu quá trình đề xuất sản phẩm nhằm tăng hiệu quả của mô hình.

4.2.2. Nhược điểm

Một trong những nhược điểm của hệ thống hiện tại là dữ liệu chưa được cập nhật theo thời gian thực, điều này ảnh hưởng trực tiếp đến tính kịp thời của phân tích. Khi dữ liệu không realtime, các quyết định đưa ra có thể dựa trên thông tin lỗi thời, không phản ánh đúng tình hình thực tế tại thời điểm hiện tại điều này ảnh hưởng tới tính chính xác của hệ thống. Ngoài ra, việc không có dữ liệu theo thời gian thực còn ảnh hưởng tới khả năng phát hiện các bất thường trong hệ thống, chẳng hạn như gian lận trong giao dịch, sự gia tăng bất thường trong lưu lượng truy cập hoặc các sự cố kỹ thuật. Một vấn đề khác là việc chậm trễ trong cập nhật dữ liệu có thể làm giảm khả năng tối ưu hóa các chiến dịch tiếp thị.

4.3. Hướng phát triển của đề tài

Để khắc phục các khuyết điểm trên**,** hướng phát triển của đề tài tập trung vào việc áp dụng dữ liệu thời gian thực (real-time data) và hệ thống gợi ý theo thời gian thực (real-time recommendation) nhằm nâng cao trải nghiệm người dùng cũng như tối ưu hóa hiệu suất bán hàng. Việc tích hợp dữ liệu thời gian thực cho phép hệ thống liên tục cập nhật và phản ánh hành vi người dùng ngay khi họ tương tác với sản phẩm, từ đó

tạo ra những đề xuất phù hợp ngay lập tức, giúp cải thiện khả năng cá nhân hóa và tăng tỷ lệ chuyển đổi.

Bên cạnh đó, nhóm chú trọng việc phát triển học máy để phân tích sâu hơn hành vi khách hàng. Áp dụng học máy cho phép hệ thống không chỉ dựa vào dữ liệu lịch sử mà còn học sâu hơn về tác động của từng biến số đến hành vi mua hàng, chẳng hạn như sự khác biệt giữa sản phẩm có giá cao hơn hay thấp hơn mức giá trung bình, mức độ ưa chuộng của các màu sắc theo thời gian, hoặc sự thay đổi xu hướng theo từng danh mục sản phẩm. Điều này giúp cải thiện đáng kể độ chính xác của mô hình dự đoán và đề xuất sản phẩm phù hợp hơn với sở thích của từng cá nhân. Hệ thống cũng có thể sử dụng các mô hình deep learning hoặc reinforcement learning để tối ưu hóa khả năng dự đoán và đưa ra quyết định theo thời gian thực, giúp giảm thiểu độ trễ và đảm bảo rằng các đề xuất luôn phản ánh đúng thị hiếu của khách hàng tại từng thời điểm. Từ đó không chỉ giúp cá nhân hóa trải nghiệm người dùng mà còn mang lại lợi ích lớn cho doanh nghiệp trong việc tăng doanh số, tối ưu hóa danh mục sản phẩm và dự đoán xu hướng tiêu dùng. Ngoài ra việc áp dụng hệ thống đề xuất sản phẩm hiển thị popup theo thời gian thực trên trang web thương mại điện tử cũng sẽ giúp tăng cường trải nghiệm người dùng và tối ưu hóa tỷ lệ chuyển đổi. Dữ liệu Clickstream sẽ được xử lý theo thời gian thực để phát hiện hành vi mua sắm và đưa ra gợi ý phù hợp bằng cách tích hợp Spark Streaming hoặc Kafka để thu thập dữ liệu liên tục, sau đó áp dụng mô hình Machine Learning hoặc thuật toán Collaborative Filtering để phân tích sở thích người dùng. Hệ thống sẽ kết nối với frontend thông qua API hoặc WebSocket để hiển thị popup gợi ý sản phẩm ngay khi người dùng duyệt web, giúp cá nhân hóa nội dung hiển thị và thúc đẩy hành vi mua hàng.